

IMU 指揮ジェスチャと発音予約同期による 分散合奏システム「タクトーン」の開発

情報工学科 2 年

25G1065

塩澤 匠生

—— チーム 23 ——

片岡 聖 (24G1038) 地曳 賢人 (24G1175)

御代川 稜 (24G1131) 梅澤 颯太 (25G1021)

齋藤 翔太 (25G1053)

2026 年 7 月 12 日

1 はじめに

合奏は複数の奏者が呼吸を合わせて一つの音楽を作る体験であるが、楽器の習熟には長い訓練を要し、経験のない人が合奏の中心である指揮者の役割を体験する機会は少ない。楽器の習熟を前提としない音楽体験の提供を目指した先行例として、センサで身体動作を捉えて発音に変換する非接触型電子楽器の開発 [1] や、コントローラを振る動作で楽器演奏を模擬する市販ゲーム [2] がある。これらは一人の操作を一つの音に変換するものが中心であり、「複数の独立した楽器を一人の指揮で同時に動かす」という合奏の構図そのものを体験させるシステムは少ない。

本プロジェクトでは、指揮者の腕の振りを慣性計測装置（IMU：Inertial Measurement Unit）で検出し、無線で接続された複数の楽器ノードを同時に演奏させる分散合奏システム「タクトーン」を製作した。システムは指揮者ノード 1 台（ESP32-S3）と楽器ノード 5 台（Arduino UNO R4 WiFi）、および音色合成を担う PC 5 台から成り、指揮者ノードが検出した拍を無線 LAN 上の UDP マルチキャストで全楽器ノードへ配信する。

このようなシステムの技術的課題は、無線通信の遅延ゆらぎ（ジッタ）の下で複数楽器の発音時刻をそろえることである。合奏では数十 ms 程度の発音時刻のずれは聴き手に気づかれにくいことが知られており [3]、本プロジェクトではチームでの議論を経て安全側の目標として楽器間同期誤差 20 ms 以内を掲げた。ネットワーク経由の分散オーケストラ演奏機構では通信遅延の補償が主要な設計課題となることが先行研究でも示されており [4]、本システムでも遅延そのものを消すのではなく遅延の影響を打ち消す設計が必要となった。

報告者はチーム内で Arduino 系プログラム全般（指揮者・楽器・共通層の設計・実装）と性能検証を担当した。本報告書では、報告者が設計・実装した拍検出、通信プロトコル、時刻同期と発音予約の各機構を中心に述べる。実装の結果、開発中の実測でアクセスポイントのマルチキャスト配送が約 205 ms 周期のバースト状になるという想定外の事実が判明し、当初計画の同期パラメータと評価指標を実測に基づいて再設計した。最終的に、楽器間の発音時刻差は中央値 7 ms（目標 20 ms の約 3 分の 1）を達成し、5 楽器の合奏が成立することを定量的に確認した。計画時の想定と最終実装の相違点、およびその変更理由についても本報告書で明示する。

2 システム全体の構成

2.1 全体構成と情報の流れ

システム構成を図 1 に示す。システムは次の 3 種類の要素から成る。

指揮者ノード（node_01） XIAO ESP32-S3 Sense に IMU（GY-521, MPU6050）を I2C 接続したマイコンである。自身が無線 LAN アクセスポイント（SoftAP）を起動し、IMU の加速度から拍を検出して、テンポ・強弱を含む制御パケット CTRL と拍パケット BEAT を UDP マルチ

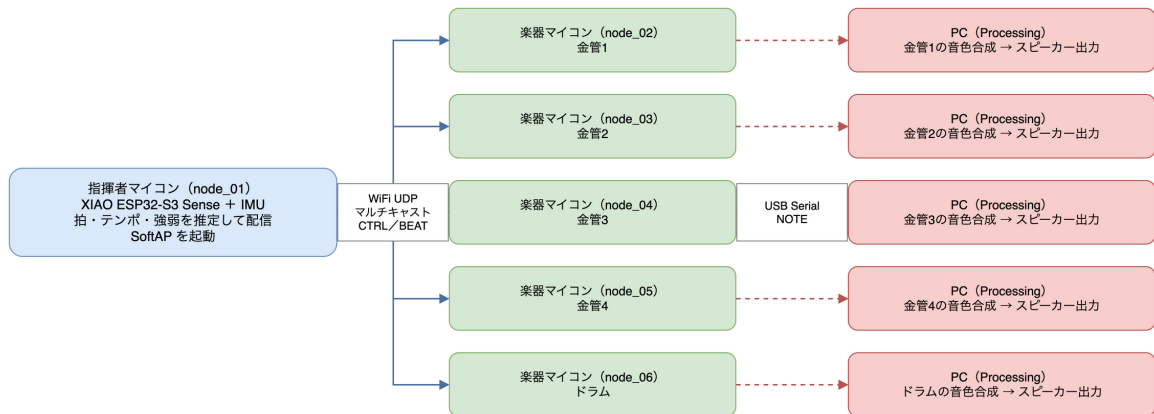


図1 システム全体構成（指揮者1台・楽器5台・PC5台）

表1 チーム23の役割分担

メンバー	学籍番号	主な担当
塩澤 匠生	25G1065	Arduino 系プログラム全般（指揮者・楽器・共通層）、性能検証
齋藤 翔太	25G1053	楽譜データ作成・音階生成ロジック・周波数分析
梅澤 颯太	25G1021	Processing による音色合成・演奏再生・UI
御代川 稜	24G1131	議事録（持ち回り1番手）、計画書編纂
地曳 賢人	24G1175	議事録（2番手）、スケジュール管理
片岡 聖	24G1038	議事録（3番手）、WBS・役割整理、機材調達

キャスト（239.0.0.1:5001）で全楽器ノードへ配信する。なお性能検証ではPCB内蔵アンテナのESP32-S3-DevKitC-1に同一ファームウェアを書き込んだ構成を用いた。

楽器ノード（node_02～06） Arduino UNO R4 WiFi[8] 5台である。金管4声部（トランペット・ホルン・フルート・オルガンの音色を割当）とドラム1台から成り、各ノードが自パートの楽譜データを保持する。BEATの予約時刻に合わせて楽譜を1拍進め、発音指示パケットNOTEをUSBシリアル（115200bps）で自分専用のPCへ送る。

PC（5台） Processing[9]製の音色合成アプリケーションがNOTEを受信し、楽器定義JSONに基づく倍音加算合成で発音する。PCは楽器ごとに1台を割り当て、スピーカも独立させることで「楽器が別々の場所で鳴る」合奏の空間性を再現する。

チーム6名の役割分担を表1に示す。報告者はArduino系プログラム全般、すなわち指揮者ノード・楽器ノード・共通ライブラリ層の設計と実装、および本報告書で述べる性能検証の全体を担当した。齋藤が楽譜データと周波数分析を、梅澤がProcessingによる音色合成・UIを担当し、報告者が設計したNOTEパケット仕様（3.2節）がファームウェアとPCアプリの境界インターフェースとなった。

2.2 動作モード

システムは自由演奏モードとゲームモードの2モードを持つ。自由演奏モードでは指揮者の振りがそのまま演奏テンポになる。ゲームモードでは目標テンポの提示とテンポ維持精度の採点を行う。モード選択は指揮者ノードのIMUナビゲーション（腕の左右振りでカーソル移動・縦振りで決定）で行い、選択状態はCTRLパケットの予約領域を用いて全ノードへ伝達される。本報告書は報告者の担当である同期・通信・拍検出を中心に述べるため、ゲームモードの採点詳細は付録のソースコードに委ねる。

3 担当部分の設計と実装

3.1 設計方針：Embedded-Module-Architecture

ファームウェアは全ノード共通で Embedded-Module-Architecture (EMA) [7] を採用した。EMA は報告者が本プロジェクト以前から整備してきた組込み向けの設計様式であり、ループを入力・ロジック・出力の3フェーズに分け、全モジュールが共有構造体 `SystemData` のみを介して通信する。モジュール同士の直接呼び出しを禁止することで、(1) 拍検出・通信・LED表示といった関心事を独立に差し替えられる、(2) 指揮者と楽器で通信・LED等のモジュールを共通ライブラリとして共有できる、(3) ロジック部が `SystemData` の読み書きだけで完結するため実機なしの机上検証がしやすい、という利点がある。実際、共通ライブラリ層（プロトコル定義・UDP送受信・受信処理・NOTE送出・LED・デバッグ出力）は7ノードのファームウェアから完全に共有され、輪唱進行ロジックの正否をPC上のPythonで再現検証することもできた（6節）。

制御周期は楽器ノードで2msとした。後述の発音予約方式では発火判定の時間分解能がそのまま同期誤差の下限になるため、当初の5msから短縮した。CPU負荷の実測（5.6節、入力フェーズ最大0.72ms）から2ms周期でも処理が間に合うことを確認している。

3.2 通信プロトコルの設計

ノード間の通信は表2に示す3種類（+UI中継1種類）の自作バイナリパケットで行う。全パケットを20バイト固定長（共通ヘッダ12バイト+ペイロード8バイト、リトルエンディアン）に統一し、先頭2バイトのマジックナンバー 0x4F52（ASCII “OR”）で境界の再同期を可能にした。固定長・単純構造としたのは、Arduino UNO R4のメモリと処理時間の制約下で、受信処理を制御周期2ms内に確実に収めるためである。パケット定義は付録のリスト2に示す。

信頼性設計として、BEATは同一内容を4連送する。UDPマルチキャストには再送がなく、拍の取りこぼしは演奏の欠落に直結するためである。同一拍番号の重複受信は楽器側で排除する。この設計の効果は評価で確認され、最終構成でのパケット欠落は5ノードすべてで0.0%であった（5.6節）。

表 2 通信パケットの種類と役割

種別	経路	頻度	主な内容
CTRL	指揮者 → 全ノード (UDP)	20 Hz	テンポ (BPM×8), 強弱, 状態, モード
BEAT	指揮者 → 全ノード (UDP)	拍ごと (4 連送)	拍番号, 発音予約時刻
NOTE	楽器 → PC (USB シリアル)	発音ごと	音高, 強弱, 音長, パート, 楽器番号
UI	楽器 → PC (USB シリアル)	変化時 (最大 5 Hz)	画面状態の中継 (ゲームモード用)

3.3 IMU による拍検出

指揮者ノードは腕の「振り下ろし」を拍として検出する。IMU の 3 軸加速度 $\mathbf{a}[n]$ (単位 g, サンプリング周期約 5 ms) に対し, まず 1 次 IIR ローパスフィルタでノイズを抑える。

$$\mathbf{a}_{\text{lpf}}[n] = (1 - \alpha) \mathbf{a}_{\text{lpf}}[n - 1] + \alpha \mathbf{a}[n], \quad \alpha = 0.10 \quad (1)$$

ここで n はサンプル番号である。次に, 起動時キャリブレーション (2 s 間の静止計測) で求めた静止時ノルム g_0 (≈ 1 g, 重力に相当) を差し引いた動加速度ノルム $d[n]$ を求める。

$$d[n] = \|\mathbf{a}_{\text{lpf}}[n]\| - g_0 \quad (2)$$

拍検出はしきい値の単純比較ではなく, 「腕が実際に動いた距離」を測るゲート式の状態機械とした。 $d[n] > 1.20$ g で武装 (Armed) 状態に入り, Armed 中のみ動加速度を二重積分して移動経路長を推定する。重力加速度を $g_a = 9.80665$ m/s², サンプル間隔を Δt , 動加速度ベクトルを $\mathbf{a}_{\text{dyn}}[n]$ (単位 g) とすると, 速度 $\mathbf{v}$ と経路長 L は次で更新される。

$$\mathbf{v}[n] = \mathbf{v}[n - 1] + g_a \mathbf{a}_{\text{dyn}}[n] \Delta t, \quad L[n] = L[n - 1] + \|\mathbf{v}[n]\| \Delta t \quad (3)$$

$L[n] \geq 0.20$ m に達した瞬間に拍を発火する。発火後も Armed を維持し, 動加速度が収まる ($d[n] < 0.20$ g, またはピーク値の 40 %未満が 40 ms 継続) までは再突入させないことで, 1 回の振りから 1 拍のみを発火させる。さらに不応期 350 ms (約 170 BPM の拍間隔に相当) を併用し, フィルタの尾引きによる二重発火を構造的に防ぐ。単純なしきい値比較ではなく経路長を併用したのは, しきい値だけでは振幅の小さい速い揺れも拍として拾いうるため, 「実際に腕が一定距離動いた」動作だけを選別する目的である。

テンポは拍間隔 $\Delta \tau_k$ (ms) から瞬時値 $b_k = 60000 / \Delta \tau_k$ を求め, 指数移動平均 (EMA: Exponential Moving Average) で平滑化する。

$$\bar{b}_k = (1 - \beta) \bar{b}_{k-1} + \beta b_k, \quad \beta = 0.30 \quad (4)$$

ここで k は拍番号であり, $\bar{b}_k$ は 40~240 BPM に制限して CTRL で配信する。 β は「1 拍の乱れ

で演奏テンポが跳ねない」程度の平滑と「テンポ変化への追従」の折衷として実機で振り心地を確かめながら決定し、テンポ追従の遅延は最大 1.2 拍と目標（2 拍以内）に収まった（5.6 節）。

3.4 時刻同期と発音予約

本システムの同期設計の核心は、「パケットの到着時刻に頼らず、発音予約と時計同期で発音時刻をそろえる」ことである。指揮者は拍を検出すると、マスタ時計（指揮者の `millis()`）で $t_{\text{lookahead}} = 220 \text{ ms}$ 先の未来時刻を発音予約時刻 `playAtMasterMs` として BEAT に載せて送る。各楽器はマスタ時計との時計オフセットを推定しておき、予約時刻が来るまで待ってから発音する。こうすることで、配送遅延が拍ごとにばらついていても、予約時刻までに届きさえすれば全楽器の発音はマスタ時計基準でそろえる。

時計オフセットの推定は次のとおりである。パケット i の送信時マスタ時刻を T_i （ヘッダの `timestampMs`）、受信時のローカル時刻を t_i とすると、観測値

$$s_i = T_i - t_i = \theta - D_i \quad (5)$$

が得られる。ここで θ は真の時計オフセット、 $D_i \geq 0$ は配送遅延である。 s_i は真値 θ から配送遅延のぶんだけ必ず下側に外れるため、直近の時間窓 W （2000 ms）内の最大値を推定値として採用する。

$$\hat{\theta} = \max_{i \in W} s_i \quad (6)$$

これは「窓内で最も配送遅延が小さかったサンプルだけを信じる」min フィルタであり、NTP 系の時刻同期で用いられる定石である。楽器は発火判定のたびに予約時刻をローカル時刻 $t_{\text{fire}} = \text{playAtMasterMs} - \hat{\theta}$ へ変換し、制御周期 2 ms で到達を判定する。発火判定の中核部を次に示す（全体は付録リスト 4）。

リスト 1 楽器ノードの発音予約発火判定（抜粋）

```

1  const int32_t targetLocalMs =
2      (int32_t)data.receiver.pending.playAtMasterMs - data.sync.offsetMs;
3  const int32_t waitMs = targetLocalMs - (int32_t)now;
4  if (waitMs <= 0) {
5      fired          = true;    // 予約時刻に到達：この周期で発音する
6      firedBeatNo    = data.receiver.pending.beatNo;
7      data.receiver.pending.valid = false;
8  }
9  // waitMs > 0: 次ループ（2ms 後）で再評価する

```

なお当初の実装では式 (6) の代わりに EMA でオフセットを平滑化しており、 $t_{\text{lookahead}}$ も 45 ms であった。これらを変更した経緯と理由は 4 節で述べる。また指揮者ノードの再起動でマスタ時計が巻き戻った場合は、 $|s_i - \hat{\theta}|$ が 1000 ms を超えた時点でフィルタを介さず即時追従（スナップ）し、発音予約を破棄して新しいマスタ時計で再開する。

表 3 計画時の想定と最終実装の相違点

項目	計画時	最終実装
発音予約の先読み時間	45 ms	220 ms
時計オフセット推定	EMA ($\alpha = 0.10$)	min フィルタ (窓 2000 ms)
MOP5 (通信遅延) の指標	片道遅延 ≤ 30 ms	発音予約の成立率へ再定義
MOP2 (音階誤差) の測定法	実音録音の周波数分析	合成アルゴリズムの忠実再現
IMU 喪失時の Fallback 遷移	自動で内蔵テンポに移行	自動遷移を無効化
成果発表会	2026 年 7 月 1 日	2026 年 7 月 8 日 (台風で延期)

3.5 楽譜進行と輪唱

楽譜は各楽器ノードが `ScoreEvent` 配列 (拍位置・ノート番号・強弱・音長) として保持し、発火した拍の指揮者拍番号 `firedBeatNo` から楽譜位置を直接算出する。経過時間ではなく拍番号で楽譜を引くため、パケットを 1 拍取りこぼしても次の拍で正しい位置に自己復帰し、ずれが蓄積しない。この「外部からの拍イベントに楽譜位置を追従させる」構成は、演奏の弾き飛ばしに追従する楽譜追跡研究 [5] の考え方を単純化して取り入れたものである。輪唱はパートごとの開始遅延 `headRestBeats` と全パート共通のサイクル窓 (曲長 + 最終声部の遅延) で管理し、最終声部が 1 周を終えるまで先頭声部が次の周回を始めない。強弱は楽譜側の `velocity` と指揮者 CTRL の `velocity` を乗算合成して NOTE に載せる。

4 計画からの変更点とその理由

計画書 (中間発表版) の想定と最終実装の主な相違点を表 3 にまとめる。いずれも開発中の実測で判明した事実に基づく変更であり、以下で個別に説明する。

4.1 バースト配送の発見と先読み時間の再設計

開発終盤の実測で、SoftAP の UDP マルチキャストが約 204.8 ms 周期のバースト状に配送されることが判明した。これは IEEE 802.11 の省電力機構 [10] に由来する挙動で、アクセスポイントはマルチキャストフレームを即時送信せず、DTIM (Delivery Traffic Indication Message) ビーコンのタイミングまでバッファリングする。その結果、BEAT の生の配送遅延は区間 $[0, 205]$ ms にほぼ一様に分布し (平均約 100 ms, 最大約 215 ms), 3 回の独立した計測すべてで同じ周期 (204.0 ~ 205.1 ms) が再現した。

当初の先読み時間 45 ms はこのバースト周期に対して構造的に不足しており、実測では 45.4 % の拍で BEAT が予約時刻までに届かず、発音予約が機能していなかった。そこで先読み時間を「バースト周期 204.8 ms + 余裕」として 220 ms へ拡大した。変更後、予約に間に合わなかった拍は 1.9 ~ 3.1 % まで減少した (5.4 節)。

なおビーコン間隔の短縮 (100 → 50 TU) によるバースト周期の短縮も試みたが、設定は受理されるもののバースト周期は 3 回の計測すべてで変わらず (204.2 ~ 205.1 ms), 効果ゼロと実証されたため撤去した。この試行は生成 AI の提案を実測で棄却した例として 6 節でも触れる。

この変更の代償として、指揮の振りから発音までの絶対遅延は約 220 ms + 発火遅刻（平均約 232 ms, 120 BPM の約半拍）の固定遅延となった。変更前は平均約 125 ms と小さいが拍ごとに暴れる遅延であった。本システムでは「遅延のばらつきを固定遅延に置き換えて楽器間の同時性を確保する」トレードオフを選択した。楽器間がそろっている限り、一定の追従遅れは指揮行為の中で自然に吸収され、成果発表会の実演でも違和感の報告はなかった。

4.2 時計同期の min フィルタ化

当初の時計オフセット推定は EMA による平滑化であった。しかしバースト配送環境では観測値 s_i が真値 θ から 0~205 ms 下側に散らばるため、EMA は平均的な位置、すなわち真値より約 40~55 ms 遅れた推定値に収束してしまう。推定時計の遅れは発音時刻の遅れに直結し、さらに遅刻量の計測値を実際より小さく見せる（対策前の計測は真の遅刻を 54~59 ms 過小表示していた）。式 (6) の min フィルタへの変更により推定時計は最小遅延のサンプルに張り付き、この系統誤差はほぼ解消した。計測値が信頼できる下限値として解釈可能になったことも、この変更の成果である。

4.3 評価指標の再定義（MOP5）

計画書の MOP5 は「指揮から楽器への通信遅延 30 ms 以内」という生の片道遅延を目標としていた。しかし実測の結果、この指標には三つの問題があることが分かった。第一に、バースト配送環境では生の配送遅延の平均が約 100 ms であり、30 ms 以下は物理的に達成不能である。第二に、片方向の時計同期では絶対片道遅延は原理的に計測できない（時計同期そのものが遅延を吸収してしまう）。第三に、本システムは発音予約方式であるから、配送遅延は予約に間に合う限り演奏品質に影響しない。つまり指標がシステムのアーキテクチャと噛み合っていなかった。

そこで MOP5 を「発音予約の成立」、すなわち受信時点の遅刻 $\text{late} = \max(0, \hat{\theta} + t_{\text{recv}} - \text{playAtMasterMs})$ と予約破綻率（遅刻した拍の割合）で測るよう再定義した。指標は下位層の物理量ではなく「アーキテクチャが何を保証する設計なのか」に対応させて定義すべきであり、これは計画段階の検証計画で気づけたはずのスコープミスであったと総括している。

4.4 音階誤差の測定方法の変更（MOP2）

計画書の MOP2 は「合成出力音を録音し周波数分析して平均律理論音高と比較する」と定義していた。しかし実音の録音にはサウンドカードの D/A 特性や部屋の音響が混入し、測りたい対象である「楽器定義 JSON と合成方法に由来する音階誤差」だけを分離できない。そこで Processing の加算合成アルゴリズムを Python へ忠実に移植し、「現在の楽器定義と発音方法で鳴るはずの波形」を数値合成して周波数解析する方法に変更した。検出される誤差が 100 % 音源データと合成式に由来することが保証され、再現も決定論的になる。推定器自体は既知信号による 10 項目のセルフテスト（純音・デチューン・ビブラート・トレモロ）で誤差 0.02 cent 未満を確認しており、測定対象の許容値 3.6 cent に対して十分な分解能を持つ。

表 4 性能指標（MOP）の目標値と実測値

指標	目標値	実測値（最終構成）	判定
MOP1 拍検出の正確性	正解率 $\geq 90\%$	100.0%（120 拍/120 拍）	達成
MOP2 音階の誤差	平均 < 3.6 cent	平均 0.737 cent・最大 1.907 cent	達成
MOP3 楽譜との相違	誤ノート 0 件	0 件	達成
MOP4 楽器間同期誤差	≤ 20 ms	中央値 7 ms・最大 65 ms	未達（実質達成）
MOP5 指揮 → 楽器の通信遅延	≤ 30 ms	予約破綻率 3.1%へ再定義	指標再定義
MOP6 テンポ追従の遅延	≤ 2 拍	最大 1.2 拍	達成
MOP7 起動時間	≤ 5 s	最大 2.3 s	達成
MOP8 CPU 負荷（入力フェーズ）	≤ 2 ms	最大 0.72 ms	達成
MOP9 パケットロス耐性	$\leq 5\%$	全ノード 0.0%	達成

4.5 Fallback 自動遷移の無効化

計画では IMU の応答喪失時に内蔵テンポで演奏を継続する Fallback 状態への自動遷移を想定していた。しかし発表前の実機テストの結果を受けて、自動遷移は無効化した。発表本番では、一時的なセンサ応答の乱れを契機に演奏状態が指揮者の意図と無関係に切り替わることを避け、システムの挙動を予見可能に保つことを優先した判断である。状態と遷移の仕組み自体はコード上に残しており、設定で再有効化できる。

5 評価

5.1 評価方法

計画書で定めた 9 項目の性能指標（MOP：Measure of Performance）を全項目実測で評価した。ファームウェアに MOP_TEST コンパイルフラグで各指標専用の計測ログ出力を組み込み、5 台の楽器ノードのシリアル出力を Python スクリプトで同時収集・自動集計する検証環境を報告者が構築した。同期系（MOP4・MOP5）は USB シリアルの到着ジッタが混入しないデバイス側計測（BEAT 受信時と発音発火時に、そのときの推定マスタ時刻を 1 行記録する方式）で測定し、2026 年 7 月 10～11 日に 3 回（対策前・対策後・最終構成、延べ 560 拍 $\times$ 5 ノード）の実測を行った。音階誤差（MOP2）は 4.4 節の方法による。

5.2 結果一覧

全 9 項目の目標値と実測値を表 4 に示す。7 項目が目標を達成し、MOP4 は基準未達ながら実質達成と判断して受け入れ、MOP5 は 4.3 節のとおり指標を再定義した。

5.3 楽器間同期誤差（MOP4）の分析

図 2 に最終構成での楽器間同期誤差を示す。同一拍における最速ノードと最遅ノードの発音時刻差は、173 拍で中央値 7 ms・平均 10.8 ms（標準偏差 12.6 ms）であり、90.8%の拍が目標の 20 ms 以

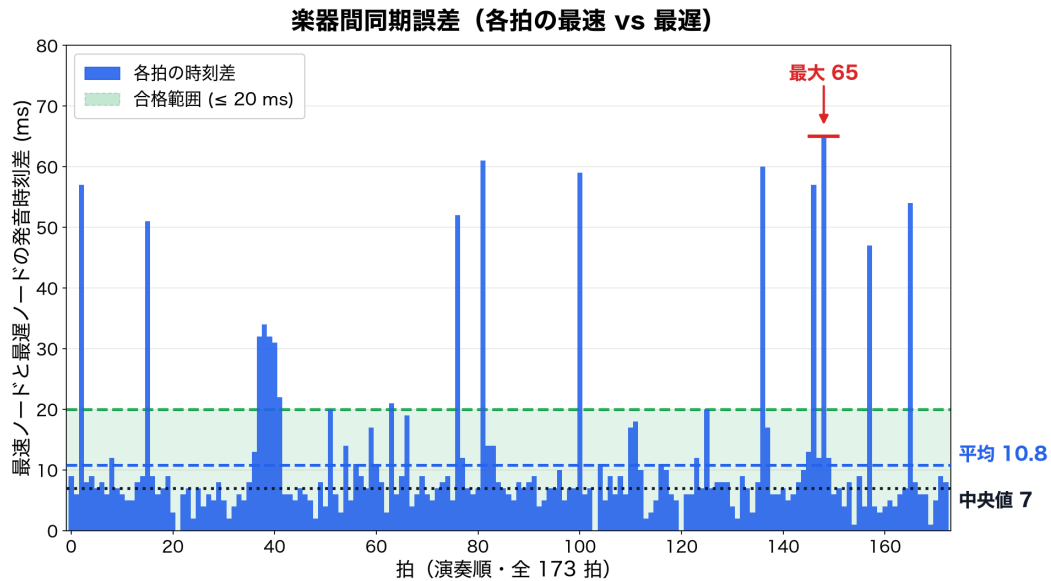


図2 楽器間同期誤差（各拍の最速ノードと最遅ノードの発音時刻差，173 拍）

内に収まった．中央値は目標値の約 3 分の 1 であり，合奏の同時性は聴感上も成立している（数十 ms のずれは気づかれにくいとされる [3]．成果発表会の実演でもチーム内外からタイミングのずれの指摘はなかった）．またノード別の平均偏差は $-1.7 \sim +3.1$ ms の範囲で，系統的に先行・遅延する楽器は存在しなかった．

一方で最大値は 65 ms であり，「全拍で 20 ms 以内」という判定基準そのものは満たせなかった．超過は 16 拍（9.2%）で，分布の裾に単発のスパイクとして現れる．原因を生ログの時系列解析で追跡した結果，楽器ノードのメインループがバースト到着から特定位相（約 120～165 ms）の区間で約 40 ms 実行されない周期的なストールが全ノードで観測され，発音予定時刻がこの区間に落ちた拍だけが 30～60 ms 遅れて発火することを特定した．ストールはビーコン設定の変更前後で不変であったことから，UNO R4 WiFi の無線モジュール（ESP32-S3 ブリッジ）内部の周期処理に由来する既存挙動と結論した．ホスト側ソフトウェアでの根治は難しく，根本対策には無線モジュール側の省電力設定の変更（モジュールファームウェアの改造）が必要である．これは本システムの現構成における限界である．

この経験から，判定基準の設計にも反省がある．ジッタを持つ通信系の品質を最大値で判定すると，中央値が目標の 3 分の 1 でも外れ値 1 拍で不合格になる．聴感品質に対応した分位点判定（例：95 パーセンタイル ≤ しきい値，または超過率 ≤ N %）を計画段階で採用すべきであった．

5.4 発音予約の成立（MOP5）と対策の効果

図 3 に発音予約方式（ジッタ吸収）の効果を示す．受信した瞬間に発音した場合（ジッタ吸収なし）の発音予定時刻とのずれは平均 101.2 ms ・最大 220 ms に達するが，予約時刻まで待って発音する実測（ジッタ吸収あり）では平均 12.1 ms ・中央値 6 ms ・最大 67 ms まで抑えられる．生の配送遅延が平均約 100 ms という環境でも合奏がそろうのはこの機構によるものであり，「届くのはバラ

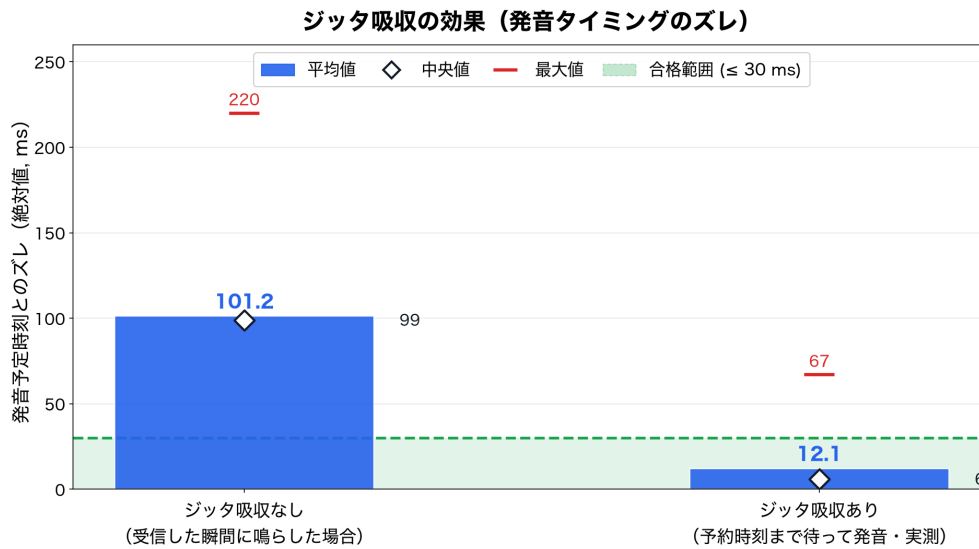


図3 発音予約方式によるジッタ吸収の効果（発音予定時刻とのずれの比較）

表5 同期対策の前後比較（2026年7月10～11日の3回の実測）

指標	対策前	対策後	最終構成
先読み時間	45 ms	220 ms	220 ms
予約破綻率（受信遅刻率）	45.4 %（465/1024 拍）	1.9 %（17/915 拍）	3.1 %（27/865 拍）
受信遅刻の最大	98 ms	14 ms	33 ms
受信マージンの平均	+2.2 ms	+102.7 ms	+100.8 ms
発火遅刻（中央値/最大）	8/101 ms	6/79 ms	6/67 ms

バラでも、「鳴るのは同時」という設計意図が定量的に裏付けられた。

対策前後の変化を表5に示す。先読み時間の拡大（45→220 ms）と時計同期の min フィルタ化により、予約破綻率は 45.4 % から 1.9～3.1 % へ減少し、破綻時の遅刻量も最大 98 ms から 14～33 ms へ縮小した。遅刻した拍も捨てずに即時発音して回復するため、演奏の欠落は生じない。残る発火遅刻の裾（95 パーセンタイルで約 47 ms）は配送でも時計同期でもなく、5.3 節と同一の周期ストールに由来することを確認しており、先読み時間をこれ以上増やしても消えない。

5.5 音階の誤差（MOP2）

図4に楽器別の音階誤差を示す。楽譜で使用する6音（C・D・E・F・G・A）を音高を持つ4楽器で合成した全24音の平均絶対誤差は0.737 cent、最大でも1.907 centであり、許容値3.6 centを最大値で判定しても満たす。誤差は音高によらずほぼ一定の系統オフセットであり、その正体は楽器定義JSONの第1倍音比 ratio が1.0でないこと（ホルン 0.9989 で -1.91 cent、トランペット 0.9995 で -0.86 cent）に完全に帰着できた。これは音源録音を周波数解析して楽器定義を作る際に生じた、宣言基音と実測第1部分音のわずかな食い違いの残留である。非調和性パラメータの寄与は +0.04～+0.08 cent で無視できる。測定は瞬時周波数法とFFTピーク法の独立2手法で実施し一致を確認した。なお本測定は合成波形のシミュレーション（4.4 節）であり、実際のD/A変換以降

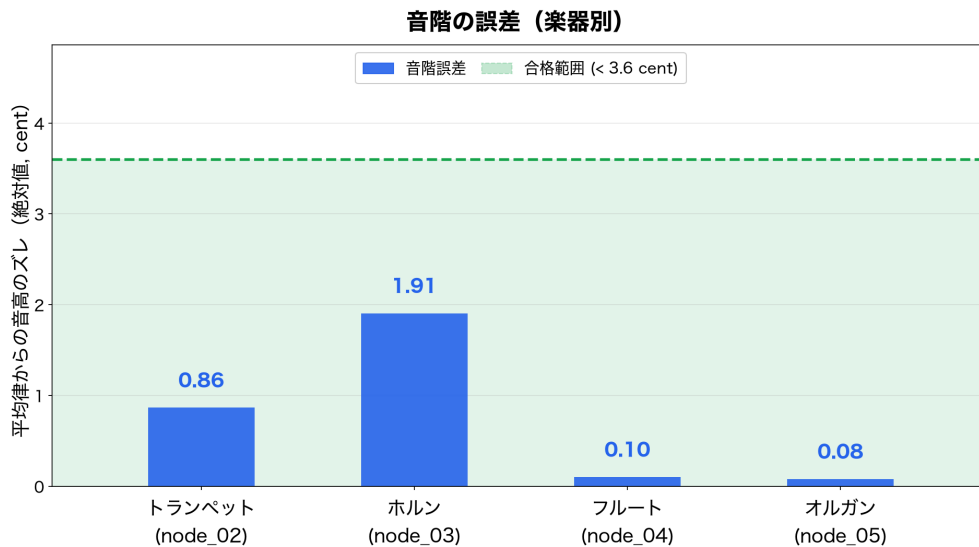


図4 音階の誤差 (楽器別の平均律からの音高のずれ. 音高を持つ4楽器が対象)

で加わる誤差は含まないが、それらは通常 cent 未満であり結論を変えない。

5.6 その他の指標

拍検出の正確性 (MOP1) は、メトロノーム 120 BPM に合わせた 120 回の振りに対して検出 120 拍・検出テンポ 119.7 BPM で正解率 100.0 %であった。楽譜との相違 (MOP3) は全パートで誤ノート 0 件である。テンポ追従 (MOP6) は 80 BPM から 130 BPM への変化に対して最大 1.2 拍の遅れで追従した。起動時間 (MOP7) は最大 2.3 s, CPU 負荷 (MOP8) は入力フェーズ平均 $71 \mu\text{s}$ ・最大 $724 \mu\text{s}$ (3 フェーズ合計でも最大 1.50 ms) で、制御周期 2 ms に対して十分な余裕がある。パケットロス耐性 (MOP9) は全 5 ノードで欠落 0.0 %・最大連続欠落 0 であった。なお MOP3 と MOP9 は開発中期の計測で一部ノードに 13.3 %の欠落と 24 件の誤ノートが出たことがあり、物理配置の変更 (ノード間距離の確保) で解消した経緯がある。誤ノートの原因がパケットロスであったことは、MOP9 の改善がそのまま MOP3 の改善に直結したことから裏付けられる。

5.7 今後の改善案

以上の分析から、改善案を 3 点挙げる。第一に、周期ストールの根本解決として無線モジュールの省電力設定の変更を検証する (ストール解消時の期待値は発火遅刻 95 パーセンタイル約 9 ms)。ループ間隔のハートビート計測機構は整備済みであり、ストールの正確な粒度の特定から着手できる。第二に、絶対片道遅延の実測には GPIO 信号とロジックアナライザによる時計非依存の計測が必要である。ただし遅延の支配項 (バースト待ち 0~205 ms) は特定済みであり優先度は低い。第三に、品質指標は聴感に対応した分位点判定で設計し直す。

6 開発支援ツールの利用と検証

開発には生成 AI（Anthropic Claude, コーディングエージェント Claude Code）を利用した。利用目的は (1) EMA に基づくモジュール雛形と共通ライブラリのコード生成, (2) 実装のレビュー（バッファ境界・整数オーバーフロー等の指摘）, (3) 検証環境（シリアル同時収集・MOP 自動集計の Python スクリプト）の作成, (4) 調査レポートやドキュメントの整備である。技術情報源としては Arduino 公式ドキュメント [8] と Processing 公式リファレンス [9] を併用した。

生成 AI の出力は無検証では採用せず、次の規律を機械的に適用した。第一に、「実機で未検証のコードに AI 起点の変更を積まない」ことをチームのルールとして明文化し、生成コードは必ずビルド・実機書き込み・動作確認を経てから採用した。これは開発初期に、机上では正しく見える拍検出の修正が実機では挙動を悪化させた経験から定めたものである。第二に、ロジックの正否は可能な限り実機によらない再現検証で確かめた。例えば輪唱進行ロジックは Python 上でサイクル窓の挙動を再現する検証スクリプトを作成し、120 拍分・20 項目の検査で「ファームウェアにバグなし・問題は実機側の構成」と切り分けた。第三に、AI の提案自体を実測で棄却した例がある。同期改善策として AI が提案した 3 案（先読み時間の拡大・ビーコン間隔の短縮・min フィルタ化）のうち、ビーコン間隔の短縮は実測でバースト周期を 1ms も変えないことを確認して撤去し、効果が実証された 2 案のみを最終構成に残した (4.1 節)。また計測系についても、AI が作成した集計スクリプトの公表値を別経路の手計算・独立集計で検算し、計測方式の不整合（受信時刻と発火時刻の混同）を発見して計測パイプラインごと再構築した。

以上のとおり、生成 AI は雛形生成と分析の加速手段として利用し、設計判断・パラメータ決定・最終的な採否はすべて実測に基づいて報告者が行った。採用したコードの内容は報告者が説明できる状態を維持しており、本報告書の記述と付録のソースコードはその実証である。

7 おわりに

本プロジェクトでは、指揮者の IMU ジェスチャで 5 台の楽器ノードと PC 音色合成を駆動する分散合奏システム「タクトーン」を製作した。報告者は Arduino 系プログラム全般（指揮者・楽器・共通層）と性能検証を担当し、次の成果を得た。

第一に、経路長積分に基づくゲート式の拍検出を設計・実装し、正解率 100.0%（120 拍/120 拍）を達成した。第二に、発音予約と時計同期による同期機構を設計・実装した。アクセスポイントのマルチキャストが約 205ms 周期のバースト配送になるという制約環境を実測で特定し、先読み時間の再設計（45→220ms）と時計同期の min フィルタ化によって、生の配送遅延が平均約 100ms の環境下でも楽器間の発音時刻差を中央値 7ms（目標 20ms の約 3 分の 1）にそろえた。「届くのはバラバラでも、鳴るのは同時」という設計原理が成立することを、延べ 560 拍 × 5 ノードの実測で定量的に示した。第三に、計画時の評価指標のうち MOP5（生の通信遅延）が本システムのアーキテクチャに対して達成不能かつ計測不能であることを明らかにし、「発音予約の成立率」へ再定

義した。指標は下位層の物理量ではなくアーキテクチャが保証する性質に対応させて定義すべきであるという教訓は、序論で述べた「遅延の影響を打ち消す設計」の評価方法として一般化できる知見である。

残された課題は、楽器ノードの無線モジュール内部に由来する周期ストール（同期誤差の裾の原因）の根本解決と、聴感に対応した分位点ベースの品質判定基準の整備である。計画との相違点はいずれも実測に基づく再設計であり、変更の理由と効果を数値で裏付けた。本システムは、無線ジッタ環境における分散合奏が予約発音方式で実用水準に到達できることを示した。

参考文献

- [1] 川合雄佑, 伊藤彰教, 「非接触型電子楽器の開発を通じたサウンドデザイン」, 先端芸術音楽創作学会会報, Vol. 14, No. 1, pp. 14–19, 2022.
- [2] 任天堂株式会社, 「Wii Music 公式サイト」, <https://www.nintendo.co.jp/wii/r64j/index.html>, 2008 (2026-07-12 閲覧) .
- [3] 河瀬諭, 「合奏における演奏者間コミュニケーション—タイミング調整とその手がかり—」, 心理学評論, Vol. 57, No. 4, pp. 495–510, 2014.
- [4] 久松剛, 「分散環境におけるフィードバックを用いたオーケストラ演奏機構の構築」, 慶應義塾大学 環境情報学部 卒業論文, 2003.
- [5] 中村栄太, 武田晴登, 山本龍一, 齋藤康之, 酒向慎司, 嵯峨山茂樹, 「任意箇所への弾き直し・弾き飛ばしを含む演奏に追従可能な楽譜追跡と自動伴奏」, 情報処理学会論文誌, Vol. 54, No. 4, pp. 1338–1349, 2013.
- [6] 竹内英世, 梅崎太造, 保黒政大, 「カラオケ採点用の高分解能ピッチ抽出法」, 電気学会論文誌 C, Vol. 129, No. 10, pp. 1889–1901, 2009.
- [7] 塩澤匠生, 「Embedded-Module-Architecture」, <https://github.com/takushio2525/Embedded-Module-Architecture> (2026-07-12 閲覧) .
- [8] Arduino, “UNO R4 WiFi Documentation”, <https://docs.arduino.cc/hardware/uno-r4-wifi/> (2026-07-12 閲覧) .
- [9] Processing Foundation, “Processing Reference”, <https://processing.org/reference/> (2026-07-12 閲覧) .
- [10] IEEE, “IEEE Standard for Information Technology—Telecommunications and Information Exchange between Systems—Local and Metropolitan Area Networks—Specific Requirements—Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications”, IEEE Std 802.11-2020, 2021.

A プログラムソース

報告者が担当したファームウェアのうち、本文で説明した中核部分のソースコードを掲載する。リスト2は全ノード共通の通信パケット定義(3.2節)、リスト3は楽器ノードの受信処理と時計同期 min フィルタ(3.4節)、リスト4は楽器ノードの発音予約発火と輪唱進行ロジック(3.5節)、リスト5は指揮者ノードの拍検出・テンポ推定・状態遷移ロジック(3.3節)である。ファームウェア全体(7ノード分の全ソース・ビルド設定・検証スクリプトを含む)はチームのGitHubリポジトリで管理している。なお組版の制約により、コメント中の記号「×」と「§」はそれぞれ「x」と「Sec.」に置換して掲載している。

リスト2 通信パケット定義 (common/lib/OrcProtocol/OrcProtocol.h)

```
1 // Build (run from project root) — shared by node_01~node_05 of production:
2 //   pio run -d firmware/production/node_01      # 指揮者ノード
3 //   pio run -d firmware/production/node_02      # 輪唱 声部 1
4 //
5 // CTRL / BEAT / NOTE / UI 共通の 20 バイト固定パケット定義
6 // 仕様: 共通ヘッダ 12 B + ペイロード 8 B、リトルエンディアン
7 //
8 // test_v2 の変更点: NotePayload の旧 reserved[0] を instrumentId に充てた。
9 // 楽器ノードは「輪唱の声部 = 楽器番号」を固定で持ち、PC 側 (
10 //   orchestra_resynth) は
11 // この番号で読み込み済みの楽器定義 (sound_lab の JSON) を選んで加算合成す
12 //   る。
13 //
14 // production の変更点 (ゲームモード):
15 //   1. CtrlPayload の旧 reserved[4] を mode/navCursor/targetBpm/score に
16 //   フィールド化。
17 //   自由演奏/ゲームのモード・メニューカーソル・目標テンポ・得点を指揮者→
18 //   楽器へ載せる。
19 //   「画面」は (state, mode) から PC が導出するのでバイトは消費しない。
20 //   2. PKT_UI (type=4) を追加。楽器ノード→PC の USB シリアル専用で UI 状態
21 //   を中継する。
22 //   UDP マルチキャストには一切流さない (同期経路 CTRL/BEAT/NOTE に無影響)
23 //   。
24 #pragma once
25 #include <stdint.h>
26 #include <stddef.h>
27
28 namespace orc {
29
30 // ASCII "OR" (リトルエンディアンで 0x52 0x4F)
31 constexpr uint16_t MAGIC = 0x4F52;
32 constexpr uint8_t  PROTOCOL_VERSION = 0x01;
```



```

28 enum PacketType : uint8_t {
29     PKT_CTRL = 1,
30     PKT_BEAT = 2,
31     PKT_NOTE = 3,
32     PKT_UI    = 4,    // production: 楽器→PC の UI 状態中継 (USB シリアル専用)
33 };
34
35 constexpr size_t HEADER_SIZE = 12;
36 constexpr size_t PACKET_SIZE = 20;
37
38 #pragma pack(push, 1)
39 struct PacketHeader {
40     uint16_t magic;        // 0x4F52
41     uint8_t  version;     // 0x01
42     uint8_t  type;        // PacketType
43     uint32_t seq;         // 単調増加
44     uint32_t timestampMs; // 送信時のマスタ時刻
45 };
46
47 struct CtrlPayload {
48     uint16_t bpmQ8;        // BPM x 8 (例: 120.5 → 964)。実振り BPM (自由演奏/ゲーム共通)
49     uint8_t  velocity;    // 0-127
50     uint8_t  state;       // 0=Idle 1=Calibrating 2=Conducting 3=Fallback 4=Menu 5=Result
51     // —— production ゲームモード: 旧 reserved[4] をフィールド化 ——
52     // 画面は (state, mode) から PC が導出するのでここには持たない。カーソルだけ別バイト。
53     uint8_t  mode;        // 0=自由演奏 / 1=ゲーム
54     uint8_t  navCursor;   // メニューカーソル位置 0..N (Menu/Result で有効)
55     uint8_t  targetBpm;   // ゲーム目標テンポ (生 BPM 40-240, 0=未設定/自由演奏では無視)
56     uint8_t  score;       // ゲーム得点 0-100, 0xFF=未確定 (Menu/Result で有効)
57 };
58
59 struct BeatPayload {
60     uint16_t beatNo;
61     uint8_t  reserved[2];
62     uint32_t playAtMasterMs; // マスタ時刻でこの ms に発音せよ
63 };
64
65 struct NotePayload {
66     uint8_t  partId;        // test_v2 は 0x02-0x04 / production は 0x02-0x05
67     // production 想定は 0x02-0x06 (ADR-0004 改訂版で楽器 5 台 = 金管 4 + ドラム 1)
68     uint8_t  noteNumber;    // MIDI 0-127, 60=C4 (高さ)
69     uint8_t  velocity;     // 0-127

```

```

69     uint8_t gate;          // 1=NoteOn, 0=NoteOff
70     uint16_t durationMs;   // 発音予定長 (長さ)
71     uint8_t instrumentId; // 0..N-1: PC 側で読み込んだ楽器定義のインデックス
                          // (楽器番号)
72     uint8_t reserved;      // 0 埋め
73 };
74
75 // production: 楽器ノード → PC への UI 状態中継ペイロード (USB シリアル専用)
76 // 。
77 // node_02 が受信 CTRL の中身を低頻度 (変化時 + 最大 5Hz) で PC へ転送し、
78 // Processing が
79 // (state, mode) から画面を自動判定する。UDP には流れない。
80 struct UiPayload {
81     uint8_t state;          // 0..5 (CtrlPayload.state と同義: Idle/
                          // Calibrating/Conducting/Fallback/Menu/Result)
82     uint8_t mode;          // 0=自由演奏 / 1=ゲーム
83     uint8_t navCursor;     // メニューカーソル位置 (Menu/Result で有効)
84     uint8_t targetBpm;     // ゲーム目標テンポ (生 BPM)
85     uint8_t score;         // ゲーム得点 0-100 / 0xFF=未確定 (Menu/Result で
                          // 有効)
86     uint8_t partId;        // 中継元の楽器ノード ID (PC の役割判定用: 0x02=メ
                          // イン操作 UI)
87     uint16_t bpmQ8;        // 実振り BPM x8 (演奏画面のテンポ表示用)
88 };
89
90 struct CtrlPacket {
91     PacketHeader header;
92     CtrlPayload payload;
93 };
94
95 struct BeatPacket {
96     PacketHeader header;
97     BeatPayload payload;
98 };
99
100 struct NotePacket {
101     PacketHeader header;
102     NotePayload payload;
103 };
104
105 struct UiPacket {
106     PacketHeader header;
107     UiPayload payload;
108 };
109 #pragma pack(pop)
110
111 static_assert(sizeof(PacketHeader) == HEADER_SIZE, "header_must_be_12_B");
112 static_assert(sizeof(CtrlPacket) == PACKET_SIZE, "ctrl_packet_must_be_20_B");

```

```

111 static_assert(sizeof(BeatPacket) == PACKET_SIZE, "beat_packet_must_be_20_B");
112 static_assert(sizeof(NotePacket) == PACKET_SIZE, "note_packet_must_be_20_B");
113 static_assert(sizeof(UiPacket) == PACKET_SIZE, "ui_packet_must_be_20_B");
114
115 // magic / version の妥当性を確認しヘッダを取り出す
116 bool parseHeader(const uint8_t* buf, size_t len, PacketHeader& out);
117
118 } // namespace orc

```

リスト 3 楽器ノードの受信処理と時計同期 (common/lib/OrcReceiverModule/OrcReceiverModule.cpp)

```

1 // Build / Upload / Monitor (run from project root):
2 //   pio run -d firmware/production/node_XX
3 //   pio run -d firmware/production/node_XX -t upload
4 //   pio device monitor -d firmware/production/node_XX
5
6 #include "OrcReceiverModule.h"
7 #include "SystemData.h"
8 #include "MopTest.h"
9
10 namespace {
11
12 // マスターリセット (offset スナップ) 検知時の後始末。
13 // pending.playAtMasterMs は旧マスタ時計の値で、新 offset では発火判定が壊れ
14 // ため破棄。
15 // lastBeatNo は新しい連番 (1 から再開) と偶然一致して初拍を重複扱いで飲む事
16 // 故を
17 // 防ぐためリセットする。performer.state を WaitStart に戻すことで、新マスタ
18 // の
19 // 最初の BEAT を受信してから演奏を再開する (リセット直後の不安定な期間を飛ば
20 // す)。
21 void resetAfterMasterReset(SystemData& data) {
22     data.receiver.pending.valid = false;
23     data.receiver.hasFirstBeat  = false;
24     data.receiver.lastBeatNo    = 0;
25     data.performer.state        = PerformerState::WaitStart;
26 }
27
28 } // namespace
29
30 // 時計同期: min フィルタ (窓内最大 offset サンプル追従、NTP 系の定石)。
31 // 返り値: スナップ (大ジャンプの即時追従) が起きたか。
32 //
33 // 設計意図 (tools/verification/results/
34 //   MOP5_systematic_shift_analysis_20260710.md Sec.4/Sec.8 案4):
35 //   SoftAP のマルチキャストは DTIM バッファリングで 204.8ms 周期のバースト配
36 //   送になり、
37 //   サンプル (= timestampMs - millis() = 真のオフセット θ - 配送遅延 D)
38 //   は

```

```

32 // 「θ から 0~205ms 下に散らばる」。旧 EMA ( $\alpha=0.20$ ) は新鮮な CTRL と
    // ~102ms 古い
33 // BEAT の平均的な位置に落ち着き、推定マスタ時計が真値より 40~55ms 遅れて
    // いた。
34 // min フィルタは配送遅延が最小のサンプル (= 最大 sample) だけを信じるた
    // め、
35 // 推定時計が最小遅延線に張り付き、この系統遅れがほぼ消える。
36 //
37 // 実装:
38 // - sample > 現 offset なら即採用 (より遅延の小さいサンプル。UNO R4 の
    // millis() が
39 // 指揮者比で遅い個体 = sample が上昇トレンドの場合の追従もこの経路で即時
    // )。
40 // - 同時に窓 (clockSyncWindowMs) 内の最大 sample を記録し、窓満了で offset
    // を
41 // 窓内最大値へ引き直す。offset が上振れノイズや下降トレンド (楽器側時計
    // が
42 // 速い個体) で古くなっても、窓長ぶんの遅れで必ず追従し直せる。
43 // - スナップ: 指揮者リセットでマスタ時計 millis() が 0 付近へ巻き戻ると
    // sample が
44 // 数十秒~数分ぶん飛ぶ。|sample - 現 offset| が snapThresholdMs を超え
    // たら
45 // フィルタをやめて即時採用し、収束カウントと窓をやり直す (旧 EMA 実装と
    // 同じ
46 // 1 パケット追従を維持)。SoftAP 直結 LAN の正常遅延 (バースト待ち含め
    // ~205ms)
47 // では閾値 1000ms に届かない。
48 bool OrcReceiverModule::updateClockOffset(SystemData& data, uint32_t
    timestampMs) {
49     const int32_t sample = (int32_t)(timestampMs - millis());
50     bool snapped = false;
51     if (data.sync.sampleCount == 0) {
52         data.sync.offsetMs = sample;
53         winValid_ = false;
54     } else {
55         const int32_t jump = sample - data.sync.offsetMs;
56         if (jump > (int32_t)cfg_.clockSyncSnapThresholdMs ||
57             jump < -(int32_t)cfg_.clockSyncSnapThresholdMs) {
58             // スナップ: 巻き戻った (または大きく飛んだ) マスタ時計に即追従
                // し、
59             // 収束カウントもやり直す。
60             data.sync.offsetMs = sample;
61             data.sync.sampleCount = 0;
62             data.sync.converged = false;
63             winValid_ = false;
64             snapped = true;
65         } else {
66             if (sample > data.sync.offsetMs) {

```

```

67         data.sync.offsetMs = sample;
68     }
69     const uint32_t nowMs = millis();
70     if (!winValid_) {
71         winValid_ = true;
72         winStartMs_ = nowMs;
73         winMaxSample_ = sample;
74     } else {
75         if (sample > winMaxSample_) winMaxSample_ = sample;
76         if (nowMs - winStartMs_ >= cfg_.clockSyncWindowMs) {
77             // 窓滿了: 窓内最大サンプルへ引き直し (下方向の再追従)、
              次の窓を開く
78             data.sync.offsetMs = winMaxSample_;
79             winValid_ = false;
80         }
81     }
82 }
83 }
84 if (data.sync.sampleCount < 0xFFFF) data.sync.sampleCount++;
85 if (data.sync.sampleCount >= cfg_.clockSyncMinSamples) data.sync.
    converged = true;
86 return snapped;
87 }
88
89 void OrcReceiverModule::updateInput(SystemData& data) {
90 #if MOP_TEST == 4 || MOP_TEST == 5
91     // ループストール検出 (ハートビート): 本モジュールの updateInput は 3
              フェーズループの
92     // 毎周期呼ばれる (名目 loopIntervalMs=2ms) ので、前回呼び出しからの間隔
              が閾値を超えたら
93     // 「ループが回っていなかった区間」として 1 行記録する。7/10-11 の再計測
              で、バースト到着から
94     // 位相 ~120~165ms の区間で発火が消える周期ストール (発火 lateMs p95 47
              ms・MOP4 尾悪化の
95     // 共通原因) が観測されており、その粒度 (単一の長ブロックか短いブロックの
              連なりか) と正確な
96     // 窓位置を確定するための計測 (MOP5_countermeasure_eval_20260710.md Sec
              .3.3/Sec.5(b)-2)。
97     // 形式: M45S,<partId>,<deviceMs>,<gapMs> (deviceMs = ストール明け = 今
              回呼び出しの millis())
98     // 閾値 10ms: 名目 2ms + 実測の健常ループ粒度 (発火 late p95 9ms) を上回
              る異常だけを拾い、
99     // シリアル帯域を守る (ストールは ~5 回/秒 なので 1 行 ~25B x 5 = 帯域の
              1% 未満)。
100 {
101     static uint32_t sLastLoopMs = 0;
102     const uint32_t nowMs = millis();
103     if (sLastLoopMs != 0) {

```

```

104         const uint32_t gapMs = nowMs - sLastLoopMs;
105         if (gapMs >= 10) {
106             mop_test::mprintf("M45S,%u,%lu,%lu\n",
107                               (unsigned)cfg_.partId,
108                               (unsigned long)nowMs,
109                               (unsigned long)gapMs);
110         }
111     }
112     sLastLoopMs = nowMs;
113 }
114 #endif
115 // 1. CTRL を受信していれば時計同期 + 状態取り込み
116 if (data.orcNet.hasNewCtrl) {
117     if (updateClockOffset(data, data.orcNet.lastCtrl.header.timestampMs))
118     {
119         resetAfterMasterReset(data);
120     }
121
122     data.ctrl.bpm = data.orcNet.lastCtrl.payload.bpmQ8 / 8.0f;
123     data.ctrl.velocity = data.orcNet.lastCtrl.payload.velocity;
124     data.ctrl.state = data.orcNet.lastCtrl.payload.state;
125     // production ゲームモード: 予約バイトから UI 状態を展開 (
126     //   UiRelayModule が PC へ中継)
127     data.ctrl.bpmQ8 = data.orcNet.lastCtrl.payload.bpmQ8;
128     data.ctrl.mode = data.orcNet.lastCtrl.payload.mode;
129     data.ctrl.navCursor = data.orcNet.lastCtrl.payload.navCursor;
130     data.ctrl.targetBpm = data.orcNet.lastCtrl.payload.targetBpm;
131     data.ctrl.score = data.orcNet.lastCtrl.payload.score;
132     data.ctrl.lastReceivedMs = millis();
133 }
134
135 // 2. BEAT 受信:
136 //   - 連送 (beatRedundancy 回) で同一 beatNo が複数届くが、payload は完
137 //     全同一。
138 //     違うのは header.timestampMs (各送信時刻) と header.seq。
139 //   - pending (発音予約) は初到着の 1 個だけ採用。同 beatNo の発火後に
140 //     後着が再キューされて「同じ拍を二度発音」する事故を避ける。
141 //   - clock sync は連送 4 個ぶん全部サンプルとして使う。min フィルタでは
142 //     重複は
143 //     自然に無害: 同一 timestampMs で millis() が経過ぶん進むため sample
144 //     は
145 //     初回以下になり、窓内最大値を押し上げない (旧 EMA の重複用  $\alpha$  は不
146 //     要になった)。
147 if (data.orcNet.hasNewBeat) {
148     const uint16_t bn = data.orcNet.lastBeat.payload.beatNo;
149     bool isDuplicate = data.receiver.hasFirstBeat &&
150                       (bn == data.receiver.lastBeatNo);

```

```

146     if (updateClockOffset(data, data.orcNet.lastBeat.header.timestampMs))
147     {
148         // この BEAT 自体は新マスタ時計の正しい playAtMasterMs を持つの
149         // で、
150         // 重複扱いを解いて「新時計の最初の BEAT」として受理し直す。
151         resetAfterMasterReset(data);
152         isDuplicate = false;
153     }
154
155     if (!isDuplicate) {
156         data.receiver.pending.valid = true;
157         data.receiver.pending.beatNo = bn;
158         data.receiver.pending.playAtMasterMs =
159             data.orcNet.lastBeat.payload.playAtMasterMs;
160         data.receiver.pending.enqueuedAtMs = millis();
161         data.receiver.lastBeatNo = bn;
162         data.receiver.hasFirstBeat = true;
163
164     #if MOP_TEST == 4 || MOP_TEST == 5
165         // MOP4/MOP5 共通の BEAT 受信記録（受理した初回のみ = 1 beat 1
166         // record）。
167         // 旧実装は main.cpp 側にも M5I 出力があり同一拍が二重記録されて
168         // いたため、
169         // 計測出力はこの 1 箇所に統一した（発火時の M45F は applyPattern
170         // .cpp）。
171         // localMasterMs = 受信処理時点の推定マスタ時刻。集計側が
172         // lateMs = max(0, localMasterMs - playAtMasterMs) を計算し、
173         // beatLookahead の発音予約が受信時点で間に合っているかを判定す
174         // る。
175         {
176             const uint32_t tLocal = millis();
177             const uint32_t localMasterMs = tLocal + (uint32_t)data.sync.
178                 offsetMs;
179             mop_test::mprintf("M45R,%u,%u,%lu,%lu,%ld,%lu\n",
180                             (unsigned)cfg_.partId, (unsigned)bn,
181                             (unsigned long)data.orcNet.lastBeat.payload
182                                 .playAtMasterMs,
183                             (unsigned long)tLocal,
184                             (long)data.sync.offsetMs,
185                             (unsigned long)localMasterMs);
186         }
187     #endif
188
189     #if MOP_TEST == 9
190         mop_test::mprintf("M9,%u,%u,%lu,%lu\n",
191                         (unsigned)cfg_.partId, (unsigned)bn,
192                         (unsigned long)data.orcNet.lastBeat.header.seq,
193                         (unsigned long)millis());
194     #endif

```



```

186     }
187     // 重複でも lastBeatMs は更新する (受信タイムアウト監視・診断ログ用)
188     data.receiver.lastBeatMs = millis();
189 }
190 }

```

リスト 4 楽器ノードの発音予約発火と輪唱進行 (node_02/src/applyPattern.cpp)

```

1 // Build / Upload / Monitor (run from project root):
2 //   pio run -d firmware/production/node_02
3 //   pio run -d firmware/production/node_02 -t upload
4 //   pio device monitor -d firmware/production/node_02
5 //
6 // 楽器ノード (輪唱の 1 声部) の判断ロジック
7 // 設計方針: BPM はあくまで補助 (durationMs 計算で使用)。score の進行は
8 // 「指揮者の拍番号 firedBeatNo」で決める。輪唱は headRestBeats だけ頭の拍を
9 // 読み飛ばしてから kScore[0] を鳴らし始める (声部ごとにずれて入る)。
10 // 周回は CANON_CYCLE_BEATS (曲長 + 最終声部の遅延) のサイクル窓で管理し、
11 // 最終声部 (node_05) が 1 周を終えるまで先頭声部は次の周回を始めない。
12 // 拍番号駆動なので、Processing をいつ起動しても「曲の現在位置」から自然に鳴
13 //     る。
14 // 処理フロー:
15 //   1. 時計オフセット更新 (受信側で済 - Receiver が data.sync を更新)
16 //   2. 演奏状態遷移 (Idle -> WaitStart -> Playing)
17 //   3. 保留 BEAT のキューイング (受信側で済 - data.receiver.pending)
18 //   4. マスタ時刻判定 (発音粒度)
19 //   5. 楽譜進行: firedBeatNo と headRestBeats から kScore のインデックスを算
20 //     出して発火
21 //   6. velocity 合成 (fireScoreEvent 内)
22 #include <Arduino.h>
23
24 #include "SystemData.h"
25 #include "ProjectConfig.h"
26 #include "SerialDebug.h"
27 #include "MopTest.h"
28 #include "score_data.h"
29
30 namespace {
31
32 // durationQ8 を ms に変換 (Q8: 256 = 1 拍)
33 // BPM は CTRL から受け取った参考値。NoteOff の予定時刻計算に使う補助。
34 uint16_t durationQ8ToMs(uint16_t durationQ8, float bpm) {
35     const float beats = (float)durationQ8 / 256.0f;
36     const float ms = beats * 60000.0f / (bpm < 10.0f ? logic_params::
37         DEFAULT_BPM : bpm);
38     return (uint16_t)(ms > 60000.0f ? 60000.0f : ms);
39 }

```

```

39 // 楽譜の 1 イベントを発火 (NoteOn 予約のみ)。
40 // 消音は Processing 側が NotePacket.durationMs から自動で行うため、ここでは
41 // noteIsSounding 等の追跡は不要。
42 // 細分音符 (subNote != 0) があれば、現在 BPM から ms を計算して予約スロット
    に積む。
43 void fireScoreEvent(SystemData& data, const ScoreEvent& ev, uint32_t now) {
44     const bool isRest = (ev.flags & 0x04) != 0 || ev.noteNumber == 0;
45     if (!isRest) {
46         // velocity 合成: score x ctrl / 127
47         uint16_t v = ((uint16_t)ev.velocity * (uint16_t)data.ctrl.velocity) /
            127;
48         if (v > 127) v = 127;
49
50         const uint16_t durMs = durationQ8ToMs(ev.durationQ8, data.ctrl.bpm);
51         data.noteOut.noteNumber = ev.noteNumber;
52         data.noteOut.velocity = (uint8_t)v;
53         data.noteOut.durationMs = durMs;
54         data.noteOut.pendingOn = true;
55     }
56
57     // 細分音符の予約 (拍頭から subOffsetQ8/256 拍ぶん遅らせて発火)
58     if (ev.subNote != 0) {
59         const float bpm = (data.ctrl.bpm >= 1.0f) ? data.ctrl.bpm
60             : logic_params::DEFAULT_BPM
61             ;
62         const float beats = (float)ev.subOffsetQ8 / 256.0f;
63         const uint32_t subDelayMs = (uint32_t)(beats * 60000.0f / bpm);
64         data.score.pendingSub = true;
65         data.score.pendingSubAtMs = now + subDelayMs;
66         data.score.pendingSubNote = ev.subNote;
67         data.score.pendingSubVelocity = ev.subVelocity;
68         data.score.pendingSubDurationMs = durationQ8ToMs(ev.subDurationQ8,
            bpm);
69     }
70
71     // 予約された細分音符の時刻が来ていれば NoteOn を出す。
72     // applyPattern の先頭で呼び、楽譜進行 (4 分 BEAT 受信) と同一ループに重なら
        ないようにする。
73     // noteOutSub (専用スロット) に書く。noteOut (メイン) とは独立なので、
74     // 同一ループ反復で fireScoreEvent が noteOut を書いても衝突しない。
75     void firePendingSub(SystemData& data, uint32_t now) {
76         if (!data.score.pendingSub) return;
77         if ((int32_t)(now - data.score.pendingSubAtMs) < 0) return;
78
79         uint16_t v = ((uint16_t)data.score.pendingSubVelocity *
80             (uint16_t)data.ctrl.velocity) / 127;
81         if (v > 127) v = 127;

```

```

82     data.noteOutSub.noteNumber = data.score.pendingSubNote;
83     data.noteOutSub.velocity    = (uint8_t)v;
84     data.noteOutSub.durationMs  = data.score.pendingSubDurationMs;
85     data.noteOutSub.pendingOn   = true;
86     DBG_PRINTF("[SUB] note=%u vel=%u dur=%u delay=%lu now=%lu\n",
87               (unsigned)data.score.pendingSubNote,
88               (unsigned)v,
89               (unsigned)data.score.pendingSubDurationMs,
90               (unsigned long)(now - data.score.pendingSubAtMs),
91               (unsigned long)now);
92     data.score.pendingSub = false;
93 }
94
95 void updatePerformerLed(SystemData& data) {
96     using namespace logic_params;
97     switch (data.performer.state) {
98         case PerformerState::Idle:
99             data.led.solidOn = false;
100             data.led.blinkIntervalMs = LED_IDLE_MS;
101             break;
102         case PerformerState::WaitStart:
103             data.led.solidOn = false;
104             data.led.blinkIntervalMs = LED_WAIT_START_MS;
105             break;
106         case PerformerState::Playing:
107             data.led.solidOn = true;
108             break;
109     }
110 }
111
112 uint32_t sLastBeatRecvMs = 0;
113
114 } // namespace
115
116 void applyPattern(SystemData& data) {
117     using namespace logic_params;
118     const uint32_t now = millis();
119
120     // 0. 予約された細分音符（8分音符等）の発火判定。
121     // 前ループまでに fireScoreEvent から積まれた予約が、現時刻に達してい
122     // れば発音する。
123     firePendingSub(data, now);
124
125     // 2. 演奏状態遷移（Idle -> WaitStart -> Playing）
126     // Playing への遷移条件は「最初の BEAT を受信した」だけ（sync.converged
127     // 待ちで
128     // 鳴らない症状を避ける）。輪唱の「頭の休符」は Playing には入ってから
129     headRestBeats

```

```

127 // ぶん拍を消費する形で表すので、ここでは入り拍を待たない。
128 switch (data.performer.state) {
129     case PerformerState::Idle:
130         if (data.orcNet.wifiConnected) {
131             data.performer.state = PerformerState::WaitStart;
132         }
133         break;
134     case PerformerState::WaitStart:
135         if (data.receiver.hasFirstBeat) {
136             data.performer.state = PerformerState::Playing;
137         }
138         // リセット復帰: BEAT 未受信でも CTRL が Conducting (state==2) なら
139         // Playing に合流する。次の BEAT が来るまで鳴らないが、到着時に即
140         // 発音できる。
141         else if (data.ctrl.state == 2 && data.ctrl.lastReceivedMs > 0) {
142             data.performer.state = PerformerState::Playing;
143             sLastBeatRecvMs = now;
144         }
145         break;
146     case PerformerState::Playing:
147         // 指揮者からの BEAT が 10 秒以上途絶えたら待機に戻して LED を点
148         // 滅に切り替える
149         if (sLastBeatRecvMs > 0 && (now - sLastBeatRecvMs) > 10000) {
150             data.performer.state = PerformerState::WaitStart;
151             data.receiver.hasFirstBeat = false;
152             sLastBeatRecvMs = 0;
153         }
154         break;
155 }
156
157 // 4. 発音判定: マスタ時刻 playAtMasterMs に揃えて発火する (本来の同期設
158 // 計)。
159 //   targetLocalMs = playAtMasterMs - sync.offsetMs (マスタ時刻 → 自分の
160 //   ローカル ms)
161 //   - waitMs <= 0 (既に到来 or 受信遅延): 即発火 (期限切れでも捨てない。
162 //   捨てると鳴らなくなる)
163 //   - waitMs > 0: この周期は何もしない。次ループで再評価し時刻到来した
164 //   ら発火する
165 // 各スレーブが同じ playAtMasterMs に対してそれぞれの sync.offsetMs を引
166 // いて
167 // 待つため、複数スレーブの発音はマスタ時刻基準で自然に揃う。
168 // 旧即発火版は受信遅延ぶんスレーブ間でズレていたが、本実装ではそのズレが
169 // 消える。
170
171 bool        fired        = false;
172 uint16_t    firedBeatNo = 0; // 発火した拍の指揮者拍番号 (1 始まり)
173 if (data.performer.state == PerformerState::Playing &&
174     data.receiver.pending.valid) {

```

```

166         const int32_t targetLocalMs =
167             (int32_t)data.receiver.pending.playAtMasterMs - data.sync.
                offsetMs;
168         const int32_t waitMs = targetLocalMs - (int32_t)now;
169         if (waitMs <= 0) {
170             fired = true;
171             firedBeatNo = data.receiver.pending.beatNo;
172 #if MOP_TEST == 4 || MOP_TEST == 5
173             // MOP4/MOP5: 発火時記録（発音予約が実際に発火した瞬間のデバイス
                側計測）。
174             // localMasterMs = 発火時点の推定マスタ時刻。同一 beatNo の
                localMasterMs の
175             // ノード間レンジが MOP4（楽器間同期誤差）、playAtMasterMs に対す
                る遅刻
176             // lateMs = max(0, localMasterMs - playAtMasterMs) が MOP5（予約
                遅刻）。
177             // USB 受信時刻に依存しない。集計は tools/verification/scripts/
                の
178             // mop4_sync_error.py / mop5_comm_delay.py。
179             {
180                 const uint32_t localMasterMs = now + (uint32_t)data.sync.
                    offsetMs;
181                 mop_test::mprintf("M45F,%u,%u,%lu,%lu,%ld,%lu\n",
                    (unsigned)ORC_RECEIVER_CONFIG.partId,
182                     (unsigned)firedBeatNo,
183                     (unsigned long)data.receiver.pending.
                        playAtMasterMs,
184                     (unsigned long)now,
185                     (long)data.sync.offsetMs,
186                     (unsigned long)localMasterMs);
187             }
188         }
189 #endif
190         data.receiver.pending.valid = false;
191         sLastBeatRecvMs = now;
192     }
193     // waitMs > 0: 次ループで再評価（pending は valid のまま残す）
194 }
195
196 // 5. 楽譜進行: 指揮者拍番号 firedBeatNo から自分の楽譜インデックスを算出
    する。
197 // 輪唱サイクル方式（4 声対応）: cyclePos = (firedBeatNo - 1) %
    CANON_CYCLE_BEATS
198 // cyclePos < headRestBeats : 自分の入り前（頭の休符）→
    鳴らさない
199 // headRest <= cyclePos < headRest+曲長 : local = cyclePos -
    headRestBeats で発火
200 // cyclePos >= headRest+曲長 : 自分の 1 周は終了 → サイ
    クル末尾まで休む

```

```

201 // サイクル長 56 拍 = 曲長 32 + 最終声部 (node_05, headRest=24) の遅延。全声部が
202 // 同じ周期を共有するため、最終声部が 1 周を終えるまで先頭声部は次の周
203 // 回を
204 // 始めない (旧 % kScoreLength 方式は各声部が勝手に周回し輪唱の終端が崩
205 // れた)。
206 // 拍番号で引くので、Processing をいつ起動しても曲の現在位置から鳴り始め
207 // る。
208 // 拍が一つ飛んでも firedBeatNo に追従するだけでズレが残らない (自己補正)
209 // 。
210 if (fired && kScoreLength > 0) {
211     const uint32_t cyclePos =
212         (uint32_t)(firedBeatNo - 1) % (uint32_t)CANON_CYCLE_BEATS;
213     const int32_t local =
214         (int32_t)cyclePos - (int32_t)ORC_RECEIVER_CONFIG.headRestBeats;
215     if (local >= 0 && (uint32_t)local < (uint32_t)kScoreLength) {
216         fireScoreEvent(data, kScore[local], now);
217         data.score.currentEventIndex = (uint16_t)local; // 診断ログ用
218     }
219 }
220 // (NoteOff 判定はない: 消音は Processing 側が NotePacket.durationMs から
221 // 自動で行う。)
222
223
224
225
226
227
228
229 // LED
230 updatePerformerLed(data);
231 }

```

リスト 5 指揮者ノードの拍検出・テンポ推定・状態遷移 (node_01/src/applyPattern.cpp)

```

1 // Build / Upload / Monitor (run from project root):
2 //   pio run -d firmware/production/node_01
3 //   pio run -d firmware/production/node_01 -t upload
4 //   pio device monitor -d firmware/production/node_01
5 //
6 // 指揮者ノードの判断ロジック
7 // 仕様 Sec.2.4.2.4 の処理フロー:
8 //   1. IIR LPF + ノルム計算
9 //   2. 拍検出 (Conducting 時のみ)
10 //   3. テンポ推定 (EMA)
11 //   4. 状態遷移 (Idle -> Calibrating -> Conducting <-> Fallback)
12 #include <math.h>
13 #include <Arduino.h>
14
15 #include "SystemData.h"
16 #include "ProjectConfig.h"
17 #include "SerialDebug.h"
18

```

```

19 namespace {
20
21 float    sLpfAcc[3] = {0, 0, 0};
22 bool     sLpfInit = false;
23 uint32_t sLastBeatMs = 0;
24 // 初期テンポ 100 BPM。1 音目 (= 最初の拍) はこの値で CTRL を流す。
25 // 2 拍目で sBpmInit が false なので「1→2 拍目の間隔」をそのまま簡易テンポと
    して採用し、
26 // 3 拍目以降は BPM_EMA_ALPHA で随時補正する。
27 float    sBpmEma = 100.0f;
28 bool     sBpmInit = false;
29
30 // 拍検出のゲートステート。
31 //   Idle   : 動加速度が小さい状態。積分しない (ノイズ蓄積なし)。
32 //   Armed  : dynNorm > BEAT_DYN_THRESHOLD_G で突入。Armed 中だけ
33 //             dynAcc を積分して経路長 sPathLen を計算する。
34 // 拍は Armed -> Idle のリリースエッジで判定する (= 振り終わりタイミング)。
35 enum class BeatGate : uint8_t { Idle, Armed };
36
37 BeatGate sGate                = BeatGate::Idle;
38 float    sVel[3]              = {0, 0, 0};    // 動加速度の時間積分 (m/s) —
    Armed 中のみ更新
39 float    sPathLen             = 0.0f;         // |sVel| の時間積分 (m) —
    Armed 中のみ更新
40 float    sArmedPeakDyn        = 0.0f;         // Armed 中の dynNorm 最大値 (デ
    バグ用)
41 uint32_t sArmedAtMs           = 0;            // Armed に入った時刻
42 uint32_t sLastImuMs          = 0;            // 直前に積分した IMU サンプル時刻
43 bool     sBeatFiredInArmed    = false;       // 現 Armed セッション中に既に発火
    したか
44 uint32_t sReleaseStartMs      = 0;            // リリース条件が連続成立し始めた
    時刻 (0=未成立)

45
46 void gateToIdle() {
47     sGate                = BeatGate::Idle;
48     sVel[0] = sVel[1] = sVel[2] = 0.0f;
49     sPathLen             = 0.0f;
50     sArmedPeakDyn        = 0.0f;
51     sArmedAtMs           = 0;
52     sLastImuMs           = 0;
53     sBeatFiredInArmed    = false;
54     sReleaseStartMs      = 0;
55 }
56
57 // テンポ推定のリセット (Menu からの演奏セッション開始時に呼ぶ)。
58 // sLastBeatMs=0 で次の拍を「1 拍目」(間隔計算なし)に戻す。これをしないと
59 // Menu 滞在時間が拍間隔として EMA に混入し、BPM_MIN クランプ値 (40) に

```

```

60 // 引っ張られた BPM を新セッションの頭で数拍引きずる。
61 void resetTempoTracking(SystemData& data) {
62     sLastBeatMs = 0;
63     sBpmEma     = 100.0f;
64     sBpmInit    = false;
65     data.tempo.bpm = 100.0f;
66     data.tempo.nextBeatPredictedMs = 0;
67 }
68
69 // Fallback に落ちる直前の状態 (= 復帰先)。Menu/Result で IMU が止まっても
70 // 復帰後に元の画面へ戻る。既定 Menu は「Calibrating 完了前に Fallback は
71 // 起きない」ため実際には使われない保険値。
72 ConductorState sStateBeforeFallback = ConductorState::Menu;
73
74 // —— production: メニューナビのゲート（拍検出ゲートとは独立） ——
75 // 重力基準の縦/横判定。加速度の遅い LPF で重力ベクトルを推定し、振り加速度
76 // (accLpf - 推定重力) を「重力軸成分」と「水平面成分」に分解する。センサに
77 // 取り付け角度がついていても「地面に対して垂直=縦（決定）/ 水平=横（カーソル
78 // になる（旧方式のセンサ軸直読みは取り付け角度で破綻した）。
79 // 瞬時値は振り始めの向きが暴れるので、Armed 中の判定窓で両成分を積算し、
80 // 窓終了かリリースの早い方で 1 回だけ判定・発火する。Menu / Result 状態での
81 // み動かす。
82 enum class NavGate : uint8_t { Idle, Armed };
83 NavGate sNavGate = NavGate::Idle;
84 uint32_t sLastNavMs = 0; // 最後に発火した時刻（不応期の基準）
85 uint32_t sNavArmedAtMs = 0; // Armed 突入時刻（判定窓の基準）
86 bool sNavFired = false; // 現 Armed セッションで発火済みか
87 float sNavVertAccum = 0.0f; // |重力軸成分| の積算
88 float sNavHorizAccum = 0.0f; // |水平面成分| の積算
89 float sNavHorizVec[3] = {0, 0, 0}; // 水平面成分ベクトルの積算（カーソル移動方向用）
90 bool sNavNeedsRelease = false; // 横振り発火後、dynNorm < NAV_RELEASE_G で解除されるまで再発火を禁止
91
92 // ナビ用の重力ベクトル推定 (accLpf をさらに遅い LPF に通したもの)。
93 // 拍検出のキャリブ値 gravityMag はスカラーなので方向の分解には使えない。
94 float sNavGrav[3] = {0, 0, 0};
95 bool sNavGravInit = false;
96
97 // 1 振りを 1 操作として処理する。横振り→カーソル移動 (data.game.navCursor を更新・false)、
98 // 縦振り→決定 (true を返す)。多重発火は Armed ゲート + 発火済みフラグ + 不
99 // 応期で防ぐ。
100 bool updateNav(SystemData& data, uint32_t now, uint8_t itemCount) {
    using namespace logic_params;
    if (!data.imu.ready) return false;

```



```

101
102     if (sNavGate == NavGate::Idle) {
103         // 横振り発火後、ユーザーが振りを止める (dynNorm < release) まで再突
            入を禁止。
104         // これにより 1 回の物理的な振りから 1 回だけ発火し、LPF 尾引きや
105         // 強制 Idle→即再 Armed で同じ振り中に 2 回トグルする問題を防ぐ。
106         if (sNavNeedsRelease) {
107             if (data.imu.dynNorm < NAV_RELEASE_G) sNavNeedsRelease = false;
108             return false;
109         }
110         // しきい値超え + 不応期経過で Armed へ (積算を開始)
111         if (data.imu.dynNorm <= NAV_SWING_THRESHOLD_G) return false;
112         if ((now - sLastNavMs) < NAV_REFRACTORY_MS) return false;
113         sNavGate = NavGate::Armed;
114         sNavArmedAtMs = now;
115         sNavFired = false;
116         sNavVertAccum = 0.0f;
117         sNavHorizAccum = 0.0f;
118         sNavHorizVec[0] = sNavHorizVec[1] = sNavHorizVec[2] = 0.0f;
119     }
120
121     // Armed 中: 振り加速度を重力軸成分と水平面成分に分解して積算する
122     const float gravNorm = sqrtf(sNavGrav[0] * sNavGrav[0] +
123                                   sNavGrav[1] * sNavGrav[1] +
124                                   sNavGrav[2] * sNavGrav[2]);
125     if (gravNorm > 1e-3f) {
126         const float invG = 1.0f / gravNorm;
127         float swing[3]; // 振り加速度ベクトル (g 単位、重力差し引き済み)
128         for (int i = 0; i < 3; ++i) swing[i] = data.imu.accLpf[i] - sNavGrav[
129             i];
130         const float vert = (swing[0] * sNavGrav[0] +
131                             swing[1] * sNavGrav[1] +
132                             swing[2] * sNavGrav[2]) * invG; // 符号付き重力
133                                                                軸成分
134         sNavVertAccum += fabsf(vert);
135         float horizSq = 0.0f;
136         for (int i = 0; i < 3; ++i) {
137             const float h = swing[i] - vert * sNavGrav[i] * invG; // 水平面
138                                                                成分
139             sNavHorizVec[i] += h;
140             horizSq += h * h;
141         }
142         sNavHorizAccum += sqrtf(horizSq);
143     }
144
145     // 判定: 窓終了かリリースの早い方で 1 回だけ発火
146     const bool windowDone = (now - sNavArmedAtMs) >= NAV_DECISION_WINDOW_MS;
147     const bool release = data.imu.dynNorm < NAV_RELEASE_G;

```

```

145     bool decide = false;
146     if (!sNavFired && (windowDone || release)) {
147         sNavFired = true;
148         sLastNavMs = now;
149         if (sNavVertAccum >= sNavHorizAccum * NAV_VERT_DOMINANCE) {
150             decide = true;    // 縦振りが支配的 → 決定
151         } else {
152             // 横振りが支配的 → カーソル移動
153             if (itemCount == 2) {
154                 // 2 項目の場合は方向に依存せずトグル。左右どちらに振っても
155                 // 毎回モードが切り替わる。方向判定は重力分解の精度や取り付け
156                 // 角度の影響で不安定になりやすく、端に張り付いて動かなくなる
157                 // 問題があったため廃止。
158                 data.game.navCursor = 1 - data.game.navCursor;
159                 sNavNeedsRelease = true;
160             } else if (itemCount > 2) {
161                 int dom = 0;
162                 for (int i = 1; i < 3; ++i) {
163                     if (fabsf(sNavHorizVec[i]) > fabsf(sNavHorizVec[dom]))
164                         dom = i;
165                 }
166                 const float dir = sNavHorizVec[dom] * NAV_LR_SIGN;
167                 if (dir < 0.0f) { if (data.game.navCursor > 0)
168                     data.game.navCursor--; }
169                 else { if (data.game.navCursor + 1 < itemCount)
170                     data.game.navCursor++; }
171                 sNavNeedsRelease = true;
172             }
173             data.sender.forceCtrlSend = true;
174         }
175         DBG_PRINTF("[N1 NAV vert=%5.2f horiz=%5.2f->%s]\n",
176             sNavVertAccum, sNavHorizAccum,
177             decide ? "DECIDE" : "CURSOR");
178     }
179     // 振りが収まったら (release) 次操作に備える。
180     // 発火済みで不応期を越えた場合は release を待たず強制的に Idle へ戻す。
181     // dynNorm が NAV_RELEASE_G と NAV_SWING_THRESHOLD_G の間に留まって Armed
182     // から
183     // 抜けられなくなり、2 回目以降の振りを検知できなくなる問題を修正。
184     if (release) {
185         sNavGate = NavGate::Idle;
186     } else if (sNavFired && (now - sLastNavMs) >= NAV_REFRACTORY_MS) {
187         sNavGate = NavGate::Idle;
188     }
189     return decide;
190 }
191
192 // — production: 状態遷移直後のデッドタイム (ジェスチャ/拍検出の不感時間)

```

```

189 // 遷移を起こした振りの残り（惰性・戻し）を次状態の入力として拾わないため、
190 // 遷移から STATE_TRANSITION_DEADTIME_MS の間は拍検出とナビを無視する。
191 ConductorState sPrevState = ConductorState::Idle;
192 uint32_t sStateEnteredMs = 0;
193
194 // 状態遷移を検知したら不感時間を開始し、拍/ナビ両ゲートの撃ちかけを破棄す
195 // 遷移は状態遷移 switch 内と拍検出ブロック内（Conducting→Result）の複数箇所
196 // 起きるため、switch の前後 2 箇所から呼ぶ（state 不変なら何もしない冪等処理
197 // ）。
198 void noteStateTransition(SystemData& data, uint32_t now) {
199     if (data.conductor.state == sPrevState) return;
200     sPrevState = data.conductor.state;
201     sStateEnteredMs = now;
202     gateToIdle();
203     sNavGate = NavGate::Idle;
204     sNavNeedsRelease = false;
205     data.beat.pathLenM = 0.0f;
206     // 状態遷移直後に CTRL を即送信し、PC 側の画面切替を高速化する
207     data.sender.forceCtrlSend = true;
208 }
209
210 bool inTransitionDeadtime(uint32_t now) {
211     return (now - sStateEnteredMs) < logic_params::
212         STATE_TRANSITION_DEADTIME_MS;
213 }
214
215 // メトロノームガイド強度（経過拍ベースの固定スケジュール・通信不要）。1.0=
216 // はっきり / 0.0=なし。
217 // 採点重みは 1 - この値（ガイドが薄い/無い区間ほど重く配点する）。
218 float gameGuideIntensity(uint16_t beatCount) {
219     using namespace logic_params;
220     if (beatCount < GAME_GUIDE_FULL_BEATS) return 1.0f;
221     if (beatCount >= GAME_GUIDE_ZERO_BEATS) return 0.0f;
222     const float span = (float)(GAME_GUIDE_ZERO_BEATS - GAME_GUIDE_FULL_BEATS)
223         ;
224     return 1.0f - (float)(beatCount - GAME_GUIDE_FULL_BEATS) / span;
225 }
226
227 void updateLed(SystemData& data) {
228     using namespace logic_params;
229     switch (data.conductor.state) {
230         case ConductorState::Idle:
231             data.led.solidOn = false;
232             data.led.blinkIntervalMs = LED_IDLE_MS;
233             break;

```

```

230     case ConductorState::Calibrating:
231         data.led.solidOn = false;
232         data.led.blinkIntervalMs = LED_CALIBRATING_MS;
233         break;
234     case ConductorState::Conducting:
235         // ゲーム中でガイドが残っている区間は目標テンポで点滅 (LED メトロ
236         // ノームガイド)。
237         // ガイドが切れたら点灯のまま (自由演奏も常時点灯)。
238         if (data.game.mode == 1 && data.game.targetBpm > 0 &&
239             gameGuideIntensity(data.game.gameBeatCount) > 0.0f) {
240             data.led.solidOn = false;
241             data.led.blinkIntervalMs = (uint16_t)(30000u / data.game.
242                 targetBpm);
243         } else {
244             data.led.solidOn = true;
245         }
246         break;
247     case ConductorState::Fallback:
248         data.led.solidOn = false;
249         data.led.blinkIntervalMs = LED_FALLBACK_MS;
250         break;
251     case ConductorState::Menu:
252         data.led.solidOn = false;
253         data.led.blinkIntervalMs = LED_MENU_MS;
254         break;
255     case ConductorState::Result:
256         data.led.solidOn = false;
257         data.led.blinkIntervalMs = LED_RESULT_MS;
258         break;
259     }
260 }
261 // namespace
262
263 void applyPattern(SystemData& data) {
264     using namespace logic_params;
265     const uint32_t now = millis();
266
267     // 1. IIR LPF + ノルム計算 + 動加速度ノルム (静止ノルム補正後) の計算
268     if (data.imu.ready) {
269         if (!sLpfInit) {
270             for (int i = 0; i < 3; ++i) sLpfAcc[i] = data.imu.acc[i];
271             sLpfInit = true;
272         } else {
273             for (int i = 0; i < 3; ++i) {
274                 sLpfAcc[i] = (1.0f - LPF_ALPHA) * sLpfAcc[i] +
275                     LPF_ALPHA * data.imu.acc[i];
276             }
277         }
278     }
279 }

```

```

276     }
277     for (int i = 0; i < 3; ++i) data.imu.accLpf[i] = sLpfAcc[i];
278     data.imu.accNorm = sqrtf(sLpfAcc[0] * sLpfAcc[0] +
279                             sLpfAcc[1] * sLpfAcc[1] +
280                             sLpfAcc[2] * sLpfAcc[2]);
281     // 動加速度ノルム = LPF 後の加速度ノルム - キャリブ済み静止ノルム (
282     //   ≡ 重力 1g)。
283     // 軸ごとではなくスカラーで引くので姿勢に依存しない。水平で校正してから
284     // 90 度傾けて振っても残留重力で誤検出しない (旧方式の不具合)。
285     // Calibrating 中は gravityMag=0 なので dynNorm=accNorm。負側は 0 ク
286     // ランプ。
287     float dynN = data.imu.accNorm - data.calibration.gravityMag;
288     if (dynN < 0.0f) dynN = 0.0f;
289     data.imu.dynNorm = dynN;
290     // dynAcc は accLpf の向きを保ったまま大きさを dynNorm にスケール
291     // (= 重力を「現在の加速度方向の成分」として差し引いた近似ベクトル)。
292     // 経路長の二重積分はこのベクトルを使う。Armed 中は accNorm が十分大
293     // きく
294     // 向きは振りに支配されるので近似として十分。
295     if (data.imu.accNorm > 1e-3f) {
296         const float k = dynN / data.imu.accNorm;
297         for (int i = 0; i < 3; ++i) data.imu.dynAcc[i] = sLpfAcc[i] * k;
298     } else {
299         for (int i = 0; i < 3; ++i) data.imu.dynAcc[i] = 0.0f;
300     }
301     // ナビ用の重力ベクトル推定: accLpf をさらに遅い LPF に通す。
302     // dynNorm が大きい間 (振り中) は更新を凍結し、振り加速度の漏れ込みを
303     // 防ぐ。
304     // 状態によらず常時回しておくことで、Menu 突入時には収束済みになって
305     // いる
306     // (Calibrating の静止 2 秒で十分収束する)。
307     if (!sNavGravInit) {
308         for (int i = 0; i < 3; ++i) sNavGrav[i] = data.imu.accLpf[i];
309         sNavGravInit = true;
310     } else if (data.imu.dynNorm < NAV_GRAV_FREEZE_G) {
311         for (int i = 0; i < 3; ++i) {
312             sNavGrav[i] += NAV_GRAV_LPF_ALPHA * (data.imu.accLpf[i] -
313             sNavGrav[i]);
314         }
315     }
316     // Armed 中のみ dynAcc を二重積分して経路長 sPathLen を更新する。
317     // Idle 中は積分しないので、起動からのノイズが蓄積することはない。
318     if (data.conductor.state == ConductorState::Conducting &&
319         data.calibration.done && sGate == BeatGate::Armed) {
320         const uint32_t sampleMs = data.imu.sampleAtMs ? data.imu.

```

```

317         sampleAtMs : now;
318     if (sLastImuMs == 0) {
319         sLastImuMs = sampleMs;
320     } else {
321         const uint32_t dtMs = sampleMs - sLastImuMs;
322         sLastImuMs = sampleMs;
323         // 5ms 周期想定。loop 遅延で dt が飛んだサンプルは捨てる。
324         if (dtMs > 0 && dtMs <= 50) {
325             const float dt = dtMs * 0.001f;
326             for (int i = 0; i < 3; ++i) {
327                 sVel[i] += data.imu.dynAcc[i] * GRAVITY_MS2 * dt;
328             }
329             const float vNorm = sqrtf(sVel[0] * sVel[0] +
330                                     sVel[1] * sVel[1] +
331                                     sVel[2] * sVel[2]);
332             sPathLen += vNorm * dt;
333         }
334         if (data.imu.dynNorm > sArmedPeakDyn) sArmedPeakDyn = data.imu.
            dynNorm;
335         data.beat.pathLenM = sPathLen;
336     } else {
337         // Conducting でない or Idle ゲート時：積分は走らせない。
338         // pathLen は前回確定値を 0 にしてデバッグの見え方を揃える。
339         data.beat.pathLenM = sPathLen;
340     }
341 }
342
343 // 4. 状態遷移
344 // 前ループの拍検出ブロック内で起きた遷移 (Conducting→Result) をここで拾
    い、
345 // Result のナビが動き出す前にデッドタイムを開始する。
346 noteStateTransition(data, now);
347 switch (data.conductor.state) {
348     case ConductorState::Idle:
349         // SoftAP 起動完了で Calibrating へ
350         if (data.orcNet.wifiConnected) {
351             data.conductor.state = ConductorState::Calibrating;
352             data.calibration.startMs = now;
353             data.calibration.sampleCount = 0;
354             data.calibration.accumNorm = 0.0f;
355             data.calibration.done = false;
356         }
357         break;
358
359     case ConductorState::Calibrating: {
360         if (data.imu.ready) {
361             // 軸ごとではなく生加速度のノルムを累積する。停止していれば姿

```

```

362     勢に
// 関係なく ≒1g になり、その平均を「静止ノルム」として保持す
// る。
363     const float n = sqrtf(data.imu.acc[0] * data.imu.acc[0] +
364                           data.imu.acc[1] * data.imu.acc[1] +
365                           data.imu.acc[2] * data.imu.acc[2]);
366     data.calibration.accumNorm += n;
367     data.calibration.sampleCount++;
368 }
369 if (now - data.calibration.startMs >= CALIBRATION_MS) {
370     if (data.calibration.sampleCount > 0) {
371         data.calibration.gravityMag =
372             data.calibration.accumNorm /
373             (float)data.calibration.sampleCount;
374     } else {
375         data.calibration.gravityMag = 1.0f;    // フォールバック:
376         1g
377     }
378     data.calibration.done = true;
379     // production: キャリブ完了後はまず Menu へ（モード選択）。
380     data.conductor.state = ConductorState::Menu;
381     data.game.mode = 0;                // 既定カーソル = 自由演奏
382     data.game.navCursor = 0;
383     data.game.targetBpm = 0;
384     data.game.score = 0xFF;
385     sNavGate = NavGate::Idle;
386 }
387 break;
388 }
389 case ConductorState::Conducting: {
390     // Fallback 遷移は無効化: 実機テストで IMU 瞬断により演奏が遮られ
391     // るため。
392     // 30 秒間拍が検出されなかったらメニューに自動復帰する
393     const uint32_t lastActivity = (sLastBeatMs > 0) ? sLastBeatMs :
394         sStateEnteredMs;
395     if ((now - lastActivity) > 30000) {
396         data.conductor.state = ConductorState::Menu;
397         data.game.mode = 0;
398         data.game.navCursor = 0;
399         data.game.targetBpm = 0;
400         data.game.score = 0xFF;
401         data.game.gameBeatCount = 0;
402         data.game.scoreErrAccum = 0.0f;
403         data.game.scoreWeightAccum = 0.0f;
404         gateToIdle();
405         resetTempoTracking(data);
406     }
407 }

```

```

405         break;
406     }
407
408     case ConductorState::Fallback:
409         // Fallback への遷移経路を全て無効化したので到達しないが、
410         // 万一入った場合は直前の状態に即復帰する。
411         data.conductor.state = sStateBeforeFallback;
412         sNavGate = NavGate::Idle;
413         break;
414
415     // —— production: メニュー (IMU ナビでモード選択。拍検出は止まる)
416     ——
417     case ConductorState::Menu: {
418         // Fallback 遷移は無効化済み (修正3)
419         if (!inTransitionDeadtme(now) && updateNav(data, now,
420             MENU_ITEM_COUNT)) {
421             if (data.game.navCursor == 1) {
422                 // ゲーム開始: 目標テンポ提示・採点カウンタをリセット
423                 data.game.mode = 1;
424                 data.game.targetBpm = GAME_TARGET_BPM;
425                 data.game.gameBeatCount = 0;
426                 data.game.scoreErrAccum = 0.0f;
427                 data.game.scoreWeightAccum = 0.0f;
428                 data.game.score = 0xFF;
429             } else {
430                 // 自由演奏
431                 data.game.mode = 0;
432                 data.game.targetBpm = 0;
433                 data.game.score = 0xFF;
434             }
435             // 演奏セッション開始: テンポ推定と拍番号を初期化する。
436             // beatNo=0 に戻すと楽器側は effective = beatNo-1-
437             // headRestBeats < 0 の
438             // 頭休符からやり直す = 毎セッション曲頭から演奏が始まる
439             // (ゲームの「32 拍 = かえるのうた 1 周を採点」の前提と一致さ
440             // せる)。
441             // 楽器側の重複排除は「bn == lastBeatNo」だけなので巻き戻しも
442             // 受理される。
443             // ※前セッションが 1 拍だけで終わった直後に限り、新セッショ
444             // ンの bn=1 が
445             // 重複と誤判定され 1 拍飲まれるが、bn=2 から自己回復するた
446             // め許容。
447             resetTempoTracking(data);
448             data.beat.beatNo = 0;
449             gateToIdle();
450             data.conductor.state = ConductorState::Conducting;
451         }
452         break;

```



```

446     }
447
448     // —— production: 結果表示（縦振り=決定で Menu へ戻る。拍検出は止ま
        る） ——
449     case ConductorState::Result: {
450         // Fallback 遷移は無効化済み（修正3）
451         if (!inTransitionDeadtme(now) && updateNav(data, now, 1)) {
452             data.conductor.state = ConductorState::Menu;
453             data.game.mode = 0;
454             data.game.navCursor = 0;
455             data.game.targetBpm = 0;
456             data.game.score = 0xFF;
457             data.game.gameBeatCount = 0;
458             data.game.scoreErrAccum = 0.0f;
459             data.game.scoreWeightAccum = 0.0f;
460         }
461         break;
462     }
463 }
464
465 // 今ループの switch 内で起きた遷移（Menu→Conducting 等）を即時反映し、
466 // 直後の拍検出ブロックがメニュー決定の振りを 1 拍目として拾わないように
        する。
467 noteStateTransition(data, now);
468
469 // 2-3. 拍検出（ゲート式ステートマシン + 早期発火）
470 //   Idle: dynNorm > BEAT_DYN_THRESHOLD_G で Armed に遷移し積分を開始
471 //   Armed:
472 //       [発火] sPathLen が BEAT_FIRE_PATH_M に達した瞬間に拍を発火
473 //           （Armed セッション中 1 回のみ、refractory も AND）。
474 //       振り始めから ~100ms で発火するので体感的に振りと同期する。
475 //       ※発火しても Armed は維持する。これが 1 振り=1 拍の鍵で、
476 //       「発火→即 Idle→次サンプルで dyn 高いまま再 Arm→
        refractory
477 //           境界で再発火」という連続発火ループを防ぐ。
478 //       [リリース] 以下のいずれかで Idle に戻して次の振りに備える
479 //           (a) dynNorm < BEAT_RELEASE_G （絶対値：完全停止）
480 //           (b) dynNorm < peak * BEAT_RELEASE_RATIO （相対値：ピークアウト）
481 //           (c) Armed 開始から BEAT_ARMED_TIMEOUT_MS 経過（保険）
482 //       未発火セッションでは pathOk も AND して弱い振りの早期リリースを
        防ぐ
483 //       （発火済セッションは pathOk が自明なので無条件）。
484 //       minHoldOk で突入直後の単発ノイズによる即リリースも防ぐ。
485 // 遷移直後のデッドタイム中は拍検出を走らせない（ゲートは遷移検知時に
        Idle 化済み）。
486 if (data.conductor.state == ConductorState::Conducting && data.imu.ready
    &&
487     !inTransitionDeadtme(now)) {

```

```

488     switch (sGate) {
489         case BeatGate::Idle:
490             if (data.imu.dynNorm > BEAT_DYN_THRESHOLD_G) {
491                 sGate = BeatGate::Armed;
492                 sArmedAtMs = now;
493                 sArmedPeakDyn = data.imu.dynNorm;
494                 sVel[0] = sVel[1] = sVel[2] = 0.0f;
495                 sPathLen = 0.0f;
496                 sLastImuMs = 0;
497                 sBeatFiredInArmed = false;
498             }
499             break;
500
501         case BeatGate::Armed: {
502             const uint32_t armedFor = now - sArmedAtMs;
503             const bool pathOk = sPathLen >= BEAT_FIRE_PATH_M;
504             const bool minHoldOk = armedFor >= BEAT_ARMED_MIN_HOLD_MS;
505
506             // —— 早期発火: 1 Armed セッションに 1 回まで ——
507             // path 閾値 + refractory 経過で発火する。Idle には戻さず
508             //   Armed を
509             // 維持し、リリース or timeout が来るのを待つ。これにより 1
510             // 振り
511             // 中の二重発火を構造的に防ぐ。
512             if (!sBeatFiredInArmed && pathOk &&
513                 (now - sLastBeatMs) >= BEAT_REFRACTORY_MS) {
514                 data.beat.event = true;
515                 data.beat.beatNo += 1;
516                 data.beat.lastBeatMs = now;
517
518                 // テンポ推定の段取り:
519                 //   1 拍目: sLastBeatMs == 0 なので何もしない (= 1 音目
520                 //     は初期値 100 BPM)
521                 //   2 拍目: sBpmInit == false なので「1→2 拍目の間隔」
522                 //     をそのまま採用 (簡易テンポ)
523                 //   3 拍目以降: BPM_EMA_ALPHA で随時補正
524                 if (sLastBeatMs != 0) {
525                     const uint32_t intervalMs = now - sLastBeatMs;
526                     if (intervalMs > 0) {
527                         const float instBpm = 60000.0f / (float)
528                             intervalMs;
529                         if (!sBpmInit) {
530                             sBpmEma = instBpm;    // 簡易テンポ確定 (1→2
531                             //   拍目)
532                             sBpmInit = true;
533                         } else {
534                             sBpmEma = (1.0f - BPM_EMA_ALPHA) * sBpmEma +

```

```

529             BPM_EMA_ALPHA * instBpm;    // 随時
530             補正
531         }
532         if (sBpmEma < BPM_MIN) sBpmEma = BPM_MIN;
533         if (sBpmEma > BPM_MAX) sBpmEma = BPM_MAX;
534         data.tempo.bpm = sBpmEma;
535         const uint32_t periodMs = (uint32_t)(60000.0f /
536             sBpmEma);
537         data.tempo.nextBeatPredictedMs = now + periodMs;
538
539         // —— production ゲーム採点 ——
540         // 実振り間隔 intervalMs と目標拍間隔の誤差を、ガ
541         // イド強度で重み付け
542         // して累積する（ガイドが薄い区間ほど重い）。1 拍
543         // 目は基準が無いので除外。
544         if (data.game.mode == 1 && data.game.targetBpm >
545             0 &&
546             data.game.gameBeatCount >= 1) {
547             const float targetInterval = 60000.0f / (
548                 float)data.game.targetBpm;
549             const float err = fabsf((float)intervalMs -
550                 targetInterval);
551             const float w = 1.0f - gameGuideIntensity(
552                 data.game.gameBeatCount);
553             data.game.scoreErrAccum += w * err;
554             data.game.scoreWeightAccum += w;
555         }
556     }
557     sLastBeatMs = now;
558     sBeatFiredInArmed = true;
559
560     // —— production ゲーム進行：経過拍を数え、規定拍に達し
561     // たら結果へ ——
562     if (data.game.mode == 1) {
563         data.game.gameBeatCount += 1;
564         if (data.game.gameBeatCount >= GAME_LENGTH_BEATS) {
565             // 得点確定：平均誤差を 0-100 へ写像（誤差小=高得
566             // 点）
567             float s = 100.0f;
568             if (data.game.scoreWeightAccum > 0.0f && data.
569                 game.targetBpm > 0) {
570                 const float avgErr =
571                     data.game.scoreErrAccum / data.game.
572                     scoreWeightAccum;
573                 const float targetInterval = 60000.0f / (
574                     float)data.game.targetBpm;
575                 const float tol = targetInterval *

```

```

564         GAME_SCORE_TOLERANCE_RATIO;
565         s = 100.0f * (1.0f - avgErr / tol);
566     }
567     if (s < 0.0f) s = 0.0f;
568     if (s > 100.0f) s = 100.0f;
569     data.game.score = (uint8_t)(s + 0.5f);
570     data.conductor.state = ConductorState::Result;
571 }
572 }
573
574 // —— リリース判定（デバウンス付き） ——
575 // 発火/未発火どちらでも評価する。
576 // 発火済なら pathOk は自明なので無条件で release/timeout で
577 // 抜ける。
578 // 未発火なら pathOk を AND して弱い振りでの早期リリースを防
579 // 止。
580 // releaseInst を即時には採用せず、BEAT_RELEASE_HOLD_MS の間
581 // 連続して成立したらリリース確定とする。これにより、振りの
582 // 最中に dynNorm が一瞬 peak*RATIO を割るだけで Armed を抜けて
583 // しまい、再 Armed→再発火が起きるチャタリングを防ぐ。
584 const bool releaseAbs = data.imu.dynNorm < BEAT_RELEASE_G;
585 const bool releaseRatio = (sArmedPeakDyn > 0.0f) &&
586     (data.imu.dynNorm <
587      sArmedPeakDyn * BEAT_RELEASE_RATIO
588      );
589 const bool releaseInst = releaseAbs || releaseRatio;
590 if (releaseInst) {
591     if (sReleaseStartMs == 0) sReleaseStartMs = now;
592 } else {
593     sReleaseStartMs = 0;
594 }
595 const bool releaseHeld = (sReleaseStartMs != 0) &&
596     (now - sReleaseStartMs >=
597      BEAT_RELEASE_HOLD_MS);
598 const bool released = minHoldOk && releaseHeld &&
599     (sBeatFiredInArmed || pathOk);
600 const bool timeout = armedFor >= BEAT_ARMED_TIMEOUT_MS;
601 if (released || timeout) {
602     const char* exitReason = timeout ? "timeout"
603                                     : releaseAbs ? "abs"
604                                     : "ratio";
605     DBG_PRINTF(" [N1_ARM_END_dur=%lu_peak=%4.2f_path=%5.3f_
606               reason=%s_fired=%s]\n",
607               (unsigned long)armedFor,
608               sArmedPeakDyn,
609               sPathLen,

```

```

605         exitReason,
606         sBeatFiredInArmed ? "YES" : "NO");
607     gateToIdle();
608     data.beat.pathLenM = 0.0f;
609 }
610     break;
611 }
612 }
613 } else if (data.conductor.state != ConductorState::Conducting) {
614     // Conducting でない状態 (Idle / Calibrating / Fallback) に入ったら
615     // ゲートを必ず Idle に戻す。
616     // ※「Conducting だが imu.ready=false」のループではここに来ない。
617     // ImuModule は 5ms 周期でサンプルするので loop の大半は ready=
        false で
618     // 通過するため、ここで gateToIdle すると Armed が次ループで毎回潰
        され、
619     // 積分が走らず ARM_END も出ない致命的バグになる。
620     if (sGate != BeatGate::Idle) gateToIdle();
621 }
622
623 // デバッグ可視化用にゲート状態を SystemData にミラー
624 data.beat.gateState = (sGate == BeatGate::Armed) ? 1 : 0;
625 data.beat.armedPeakDyn = sArmedPeakDyn;
626
627 // LED 状態
628 updateLed(data);
629 }

```